

强化学习：从 MDP 到 PPO

目录

第一部分 强化学习基础	1
1 MDP 与强化学习问题	1
2 Return 与 Bellman expectation	2
3 Bellman optimality 与最优 policy	3
4 Value iteration 与 policy iteration	4
5 Monte Carlo 方法	6
6 Stochastic approximation 与 SGD	7
7 TD learning	9
8 SARSA 与 Q-learning	10
9 Function approximation 与 DQN	12
第二部分 Policy Gradient 与 Actor-Critic	15
10 Policy gradient 的目标	15
11 Policy gradient theorem 与 REINFORCE	16
12 Actor-Critic 与 advantage	18
13 Importance sampling 与 off-policy	20
14 Deterministic policy gradient	21
第三部分 TRPO: 理论与优化	22
15 TRPO 概述	22
16 Performance Difference Lemma	22
17 Surrogate 与性能下界	23
18 KL 约束与局部近似	25
19 更新方向、CG 与 HVP	26

第四部分 PPO: 目标函数与梯度	28
20 从 TRPO 到 PPO	28
21 Adaptive KL penalty	29
22 PPO-clip	30
23 PPO 的梯度传播	31
24 Clipping 的分段梯度	32
第五部分 PPO 实现: stable-baselines3	33
25 完整 objective	33
26 Rollout collection	35
27 RolloutBuffer 与 ReplayBuffer	36
28 GAE	39
29 训练循环与 schedules	42
30 Action evaluation	43
31 Advantage normalization 与 ratio	44
32 Policy loss 与 value clipping	45
33 KL early stopping	46
34 KL 的 Monte Carlo estimation	47
附录: 来源、索引与校订	48
A 来源与整理说明	48
B 手写笔记索引与校订	49
C PPO 实现页码索引	50

第一部分 强化学习基础

符号约定： 随机变量用 S_t, A_t, R_{t+1} ，观测值用 s_t, a_t, r_{t+1} ； v_π, q_π 表示真实 value functions， v_k, q_k 表示迭代估计， $\hat{v}(s, w), \hat{q}(s, a, w)$ 表示参数化近似。除 average reward 部分外，默认 $0 \leq \gamma < 1$ 、reward 有界；矩阵和求和证明先在有限 state/action spaces 下讨论。真正终止状态的后续 value 取零；时间上限截断与真正终止不应混淆。

[返回完整目录](#)

1 Reinforcement Learning 与 Markov Decision Process

对应原笔记第 1 页。

Reinforcement Learning 的目标是通过与环境交互，找到使长期累计 reward 尽可能大的 policy。学习可以利用已知的环境 model，也可以利用采样得到的 experience 和统计估计。

1.1 统计估计与 incremental update

对随机变量 X ，可用样本平均估计期望：

$$\mathbb{E}[X] \approx \frac{1}{N} \sum_{i=1}^N X_i. \quad (1)$$

Non-incremental 方法直接对已有样本汇总；incremental 方法在旧估计上利用新样本递推更新，不必每次重新计算全部历史数据。

1.2 MDP 的组成与一步交互

Markov Decision Process (MDP) 包含 state space $\mathcal{S}$ 、action space $\mathcal{A}(s)$ 、transition model、reward model。Policy 决定如何选择 action。Markov property 表示：给定当前 state 和 action 后，下一步的环境反馈不再依赖更早的历史。

$$S_t \xrightarrow{A_t} (R_{t+1}, S_{t+1}). \quad (2)$$

笔记分别写出三个条件分布：

$$\pi(a | s) = \Pr(A_t = a | S_t = s), \quad (3)$$

$$p(r | s, a) = \Pr(R_{t+1} = r | S_t = s, A_t = a), \quad (4)$$

$$p(s' | s, a) = \Pr(S_{t+1} = s' | S_t = s, A_t = a). \quad (5)$$

Policy 属于 agent，后两个分布描述 environment。分别列出 reward 和 transition 的边缘分布，并不意味着二者条件独立。记期望即时 reward 为

$$r(s, a) = \mathbb{E}[R_{t+1} | s, a] = \sum_r r p(r | s, a). \quad (6)$$

原稿“已知三个 models”的批注对应后续 model-based 推导，不表示所有 Reinforcement Learning 问题都预先知道环境模型。

2 Return、Value Functions 与 Bellman Expectation Equation

对应原笔记第 2-3 页，Lecture 2。

2.1 从 trajectory 到 discounted return

多步 trajectory 为 $S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}, R_{t+2}, \dots$ 。Discounted return 是随机变量：

$$G_t = \sum_{j=0}^{\infty} \gamma^j R_{t+j+1} = R_{t+1} + \gamma G_{t+1}. \quad (7)$$

State value 是从 state s 开始、之后遵循 π 的期望 return：

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]. \quad (8)$$

2.2 Bellman expectation equation 的展开

先对 action 求条件期望：

$$r_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} \mid S_t = s] = \sum_a \pi(a \mid s) r(s, a), \quad (9)$$

$$p_{\pi}(s' \mid s) = \sum_a \pi(a \mid s) p(s' \mid s, a). \quad (10)$$

再用 Markov property 和全期望公式：

$$\mathbb{E}_{\pi}[G_{t+1} \mid S_t = s] = \sum_{s'} p_{\pi}(s' \mid s) v_{\pi}(s'). \quad (11)$$

因此

$$v_{\pi}(s) = r_{\pi}(s) + \gamma \sum_{s'} p_{\pi}(s' \mid s) v_{\pi}(s') \quad (12)$$

$$= \sum_a \pi(a \mid s) \left[r(s, a) + \gamma \sum_{s'} p(s' \mid s, a) v_{\pi}(s') \right]. \quad (13)$$

每个 state 对应一个方程；求解整个方程组称为 policy evaluation。

2.3 矩阵形式与迭代求解

设 $\mathcal{S} = \{s_1, \dots, s_n\}$ ，定义

$$v_{\pi} = \begin{bmatrix} v_{\pi}(s_1) \\ \vdots \\ v_{\pi}(s_n) \end{bmatrix}, \quad r_{\pi} = \begin{bmatrix} r_{\pi}(s_1) \\ \vdots \\ r_{\pi}(s_n) \end{bmatrix}, \quad (P_{\pi})_{ij} = p_{\pi}(s_j \mid s_i). \quad (14)$$

于是

$$v_\pi = r_\pi + \gamma P_\pi v_\pi, \quad v_\pi = (I - \gamma P_\pi)^{-1} r_\pi. \quad (15)$$

也可从任意有限初值迭代:

$$v_{k+1} = r_\pi + \gamma P_\pi v_k. \quad (16)$$

令 $e_k = v_k - v_\pi$, 则

$$e_{k+1} = \gamma P_\pi e_k, \quad e_k = \gamma^k P_\pi^k e_0, \quad \|e_k\|_\infty \leq \gamma^k \|e_0\|_\infty \rightarrow 0. \quad (17)$$

这里使用了 stochastic matrix 的每行元素非负且和为 1。

2.4 Action value 与 state value

Action value 固定第一步 action, 而后遵循 policy:

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] \quad (18)$$

$$= r(s, a) + \gamma \sum_{s'} p(s' \mid s, a) v_\pi(s'), \quad (19)$$

$$v_\pi(s) = \sum_a \pi(a \mid s) q_\pi(s, a). \quad (20)$$

已知 q_π 和 π 可得到 v_π ; 已知 v_π 并掌握环境模型时可得到 q_π 。校订说明: 原稿展开即时 reward 的一处写成 G_{t+1} , 个别后续项漏写 γ ; 此处按 return 定义修正。

3 Bellman Optimality Equation 与最优 Policy

对应原笔记第 3-4 页, Lecture 3。

3.1 最优性与 contraction

若对所有 state 和 policy 都有 $v_{\pi^*}(s) \geq v_\pi(s)$, 则 π^* 为 optimal policy。对候选向量 v , 先定义

$$q_v(s, a) = r(s, a) + \gamma \sum_{s'} p(s' \mid s, a) v(s'). \quad (21)$$

Bellman optimality operator 为

$$(Tv)(s) = \max_{\pi(\cdot \mid s)} \sum_a \pi(a \mid s) q_v(s, a) = \max_a q_v(s, a). \quad (22)$$

向量形式为 $Tv = \max_\pi (r_\pi + \gamma P_\pi v)$, 最大值按 state 分别取得。Bellman Optimality Equation (BOE) 即 $v = Tv$ 。利用 $|\max_a x_a - \max_a y_a| \leq \max_a |x_a - y_a|$,

$$|(Tv)(s) - (Tu)(s)| \leq \gamma \max_a \left| \sum_{s'} p(s' \mid s, a) (v(s') - u(s')) \right| \quad (23)$$

$$\leq \gamma \|v - u\|_\infty. \quad (24)$$

所以 T 是 contraction, 有唯一 fixed point v^* , 且 $v_{k+1} = Tv_k \rightarrow v^*$ 。

3.2 Fixed point 的最优性证明

选择关于 v^* 的 greedy policy π^* , 则

$$v^* = Tv^* = r_{\pi^*} + \gamma P_{\pi^*} v^*. \quad (25)$$

固定 policy 的 Bellman equation 解唯一, 所以 $v^* = v_{\pi^*}$ 。另一方面, 对任意 π ,

$$v^* \geq r_{\pi} + \gamma P_{\pi} v^*, \quad v^* - v_{\pi} \geq \gamma P_{\pi}(v^* - v_{\pi}) \geq \gamma^m P_{\pi}^m(v^* - v_{\pi}). \quad (26)$$

最后一项趋于零, 故 $v^* \geq v_{\pi}$ 。固定一种并列最大值选择规则后, 可取 deterministic optimal policy:

$$a^*(s) \in \arg \max_a q^*(s, a), \quad \pi^*(a | s) = \begin{cases} 1, & a = a^*(s), \\ 0, & \text{otherwise,} \end{cases} \quad (27)$$

其中 $q^*(s, a) = r(s, a) + \gamma \sum_{s'} p(s' | s, a) v^*(s')$ 。

3.3 Reward 的 affine transformation

在无限时域 discounted 设定下, 若每一步 $R'_{t+1} = cR_{t+1} + b$, 则

$$G'_t = cG_t + \frac{b}{1-\gamma}, \quad v'_\pi(s) = cv_\pi(s) + \frac{b}{1-\gamma}. \quad (28)$$

当 $c > 0$, value 排序和最优 policy 不变。**校订说明:** 原稿的 $c \neq 0$ 应为 $c > 0$; 负数缩放会反转排序。若只对长度依赖 policy 的有限 episode 中终止前的 reward 加常数, 附加项依赖 episode 长度, 不能直接套用此结论。

4 Value Iteration、Policy Iteration 与截断评估

对应原笔记第 5-6 页, Lecture 4。

4.1 Value iteration: greedy update 与一次 backup

给定 v_k , 计算

$$q_k(s, a) = r(s, a) + \gamma \sum_{s'} p(s' | s, a) v_k(s'), \quad \pi_{k+1} \in \text{Greedy}(q_k). \quad (29)$$

再做一步 value backup:

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k = Tv_k. \quad (30)$$

v_k 只是迭代近似, 不一定等于某个 policy 的真实 state value。

4.2 Policy iteration: 充分评估, 再改进

Policy iteration 从 π_0 出发, 交替执行:

1. Policy evaluation: 求解 $v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$ 。
2. Policy improvement: 计算 q_{π_k} , 取 $\pi_{k+1} \in \text{Greedy}(q_{\pi_k})$ 。

令 $T_{\pi}v = r_{\pi} + \gamma P_{\pi}v$, 则

$$T_{\pi_{k+1}}v_{\pi_k} = Tv_{\pi_k} \geq T_{\pi_k}v_{\pi_k} = v_{\pi_k}. \quad (31)$$

继续在新 policy 下 evaluation, 得到 $v_{\pi_{k+1}} \geq v_{\pi_k}$ 。在有限 discounted MDP 中, 采用一致的并列选择规则, exact policy iteration 可达到最优 policy。

原截图算法的可编辑转写:

- 1: 给定全部 (s, a) 的 reward/transition model, 初始化 π_0 。
- 2: **for** $k = 0, 1, 2, \dots$ **do**
- 3: 初始化 evaluation 向量 u 。
- 4: **repeat**
- 5: 对所有 state 同步更新 $u \leftarrow r_{\pi_k} + \gamma P_{\pi_k}u$ 。
- 6: **until** 达到 evaluation 精度要求
- 7: 计算 $q(s, a) = r(s, a) + \gamma \sum_{s'} p(s' | s, a)u(s')$ 。
- 8: $\pi_{k+1} \leftarrow \text{Greedy}(q)$ 。
- 9: **if** policy 不再变化 **then**
- 10: 停止迭代。
- 11: **end if**
- 12: **end for**

理论上的 exact evaluation 与实际代码的有限容差应区分。没有 model 时不能直接使用上述求和; 下一节改用 $q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | s, a]$ 的样本估计。

4.3 Truncated policy iteration

每次只进行固定次数的 evaluation。给定 π_k, v_{k-1} , 令

$$v_k^{(0)} = v_{k-1}, \quad v_k^{(j+1)} = T_{\pi_k}v_k^{(j)}, \quad j = 0, \dots, m-1, \quad (32)$$

再取 $v_k = v_k^{(m)}$ 并进行 greedy improvement。

- 1: 初始化 π_0, v_{-1} , 给定截断步数 $m \geq 1$ 。
- 2: **for** $k = 0, 1, 2, \dots$ **do**
- 3: $u \leftarrow v_{k-1}$ 。
- 4: **for** $j = 1, \dots, m$ **do**
- 5: $u \leftarrow r_{\pi_k} + \gamma P_{\pi_k}u$ 。

```

6:   end for
7:    $v_k \leftarrow u$ 。
8:    $q_k(s, a) \leftarrow r(s, a) + \gamma \sum_{s'} p(s' | s, a) v_k(s')$ 。
9:    $\pi_{k+1} \leftarrow \text{Greedy}(q_k)$ 。
10: end for

```

原稿强调：有限步 evaluation 得到的 v_k 不一定是 v_{π_k} 。在标准 discounted、精确同步 backup 的设定下，此方法仍可收敛；它减少了每轮 policy evaluation 的计算量。

5 Monte Carlo Policy Evaluation 与 Control

对应原笔记第 6–8 页，Lecture 5。

5.1 Monte Carlo basic

从 (s, a) 开始，后续遵循 π_k ，得到 episode return $G^{(i)}(s, a)$ 。收集 N 个样本，用

$$\hat{q}_{\pi_k}(s, a) = \frac{1}{N} \sum_{i=1}^N G^{(i)}(s, a) \quad (33)$$

估计 action value，再据此更新 greedy policy。

```

1: 初始化  $\pi_0$ 。
2: for  $k = 0, 1, 2, \dots$  do
3:   for 每个  $(s, a)$  do
4:     生成若干从  $(s, a)$  开始、随后遵循  $\pi_k$  的 episodes。
5:     对 discounted returns 取平均，得到  $\hat{q}_{\pi_k}(s, a)$ 。
6:   end for
7:    $\pi_{k+1} \leftarrow \text{Greedy}(\hat{q}_{\pi_k})$ 。
8: end for

```

对每个 pair 单独采集大量 episodes，效率较低。Episodic task 可等到真正终止；continuing discounted task 实际常用较长的有限轨迹近似无限 return，截断会产生尾项误差。

5.2 Monte Carlo Exploring Starts

Exploring Starts (ES) 要求每个允许的初始 (s, a) 都有正概率被选到。一个 episode 也可用于更新沿途访问的 pairs。First-visit MC 只使用每个 pair 在该 episode 中首次出现时的 return；every-visit MC 使用每次出现。

```

1: 初始化  $q, \pi$ ，每个  $\text{Returns}(s, a)$  为空列表。
2: for 每个 episode do
3:   按 Exploring Starts 选择  $(s_0, a_0)$ ，之后按  $\pi$  生成至终止时刻  $T$ 。

```

```

4:    $G \leftarrow 0$ 。
5:   for  $t = T - 1, T - 2, \dots, 0$  do
6:      $G \leftarrow r_{t+1} + \gamma G$ 。
7:     if  $(s_t, a_t)$  未在  $(s_0, a_0), \dots, (s_{t-1}, a_{t-1})$  中出现 then
8:       将  $G$  追加到  $\text{Returns}(s_t, a_t)$ 。
9:        $q(s_t, a_t) \leftarrow \text{mean}(\text{Returns}(s_t, a_t))$ 。
10:      在  $s_t$  处立即将  $\pi$  更新为关于  $q$  的 greedy policy。
11:    end if
12:  end for
13: end for

```

“一个 episode 复用” “即刻更新” 分别对应轨迹复用和每次 value 更新后的 policy improvement。截图中 Returns 加入 G 应理解为追加样本，而不是不记录计数地相加。

5.3 Epsilon-soft 与 epsilon-greedy

实际环境未必允许任意指定起点，可通过持续探索获取数据。令 $m_s = |\mathcal{A}(s)|$ ，任选 greedy action $a^*(s)$ ：

$$\pi_\epsilon(a | s) = \begin{cases} 1 - \epsilon + \epsilon/m_s, & a = a^*(s), \\ \epsilon/m_s, & a \neq a^*(s). \end{cases} \quad (34)$$

$\epsilon = 0$ 是 greedy policy， $\epsilon = 1$ 是 uniform policy，体现 exploitation 与 exploration 的权衡。原稿建议较小的 ϵ ，意在保留探索的同时偏重当前较优 action。

将 MC control 中的 greedy improvement 改为 ϵ -greedy improvement，再持续采样即可。固定 $\epsilon > 0$ 时，考虑的是保留探索约束的 policy。严格说，满足 $\pi(a | s) \geq \epsilon/m_s$ 的集合是 ϵ -soft policy 集合，其对给定 q 的 greedy 改进取式 (34) 的形式。

6 Stochastic Approximation 与 Stochastic Gradient Descent

对应原笔记第 8–10 页。

6.1 样本均值的递推

设 $m_k = k^{-1} \sum_{i=1}^k X_i$ ，则

$$m_k = \frac{k-1}{k} m_{k-1} + \frac{1}{k} X_k = m_{k-1} + \frac{1}{k} (X_k - m_{k-1}). \quad (35)$$

用一般 step size 替换 $1/k$ ，得到

$$w_{k+1} = w_k - \alpha_k (w_k - X_k). \quad (36)$$

即旧估计加上对新样本误差的修正；这里统一了原稿错开一位的均值下标。

6.2 Robbins–Monro root finding

用带噪观测求解 $g(w) = 0$ 。设 $\tilde{g}(w_k, \eta_k) = g(w_k) + \eta_k$ ，则

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k). \quad (37)$$

原稿列出正斜率、step size 和零均值 noise 条件。一种标量充分条件组合为：存在根 w^* ，有限初值，且

$$0 < c_1 \leq g'(w) \leq c_2 < \infty, \quad (38)$$

$$\alpha_k > 0, \quad \sum_k \alpha_k = \infty, \quad \sum_k \alpha_k^2 < \infty, \quad (39)$$

$$\mathbb{E}[\eta_k | \mathcal{H}_k] = 0, \quad \mathbb{E}[\eta_k^2 | \mathcal{H}_k] \leq C < \infty. \quad (40)$$

$\mathcal{H}_k$ 是截至第 k 步的历史信息。在这类条件下有 $w_k \rightarrow w^*$ 的 almost sure convergence。校订说明：原稿只写每步二阶矩有限；这里补上统一的 noise 二阶矩控制，避免把过弱条件当成完整定理。均值估计的例子为

$$g(w) = w - \mathbb{E}[X], \quad w - X = g(w) + \underbrace{\mathbb{E}[X] - X}_{\eta}, \quad w^* = \mathbb{E}[X]. \quad (41)$$

6.3 从 root finding 到 SGD

考虑 $J(w) = \mathbb{E}[f(w, X)]$ 。当微分与期望可交换时， $\nabla J(w) = \mathbb{E}[\nabla_w f(w, X)]$ ，故可用单样本 gradient 更新：

$$w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, X_k). \quad (42)$$

$f(w, X) = \frac{1}{2}(w-X)^2$ 时，SGD 退化为均值更新。在 strong convexity、smoothness 和相应 noise/step size 条件下，SGD 是求解 $\nabla J(w) = 0$ 的 stochastic approximation。多维情况下，原稿二阶导数的上下界对应 Hessian 的正定上下界。

6.4 接近最优点时的相对 noise

标量情况下定义

$$\zeta_k = \frac{|f'_w(w_k, X_k) - J'(w_k)|}{|J'(w_k)|}, \quad J'(w_k) \neq 0. \quad (43)$$

若 $J'(w^*) = 0$ ，mean value theorem 给出某个介于 w_k, w^* 间的 ξ_k ，使

$$J'(w_k) = J''(\xi_k)(w_k - w^*). \quad (44)$$

若 $J'' \geq c > 0$ ，则

$$\zeta_k \leq \frac{|f'_w(w_k, X_k) - J'(w_k)|}{c|w_k - w^*|}. \quad (45)$$

原稿的直观解释是：noise 尺度若未同步缩小，远离最优点时真实 gradient 较明显，接近最优点时随机扰动相对更突出。此上界不能单独证明相对误差必然增大；在 w^* 处分母为零，指标也不再定义。

6.5 BGD、minibatch 与 SGD

对 $J_N(w) = N^{-1} \sum_{i=1}^N f(w, x_i)$, BGD 每次用全部样本, minibatch 用 m 个样本平均, 单样本 SGD 对应 $m = 1$ 。原稿 “ $m = N$ 未必等于 BGD” 适用于有放回随机采样: 可能有重复和遗漏。若无放回地取完整数据集, 则就是 full-batch gradient。

7 Temporal-Difference Learning 与 State Value 估计

对应原笔记第 10–12 页。

7.1 TD target 与逐状态更新

给定 π , 利用单步 experience (s_t, r_{t+1}, s_{t+1}) 估计 v_π 。原稿先用 prediction 减 target 定义 TD error:

$$y_t = r_{t+1} + \gamma v_t(s_{t+1}), \quad e_t = v_t(s_t) - y_t. \quad (46)$$

更新为

$$v_{t+1}(s) = \begin{cases} v_t(s_t) - \alpha_t(s_t)e_t, & s = s_t, \\ v_t(s), & s \neq s_t. \end{cases} \quad (47)$$

其他 state 不变。固定本步 y_t 时,

$$v_{t+1}(s_t) - y_t = (1 - \alpha_t(s_t))(v_t(s_t) - y_t). \quad (48)$$

若 $0 < \alpha_t(s_t) \leq 1$, 当前估计移向当前 target; 但 target 随学习改变, 这个关系不是完整的收敛证明。

7.2 Bellman residual 不等于 sample TD error

对真实 value 定义

$$e_{\pi,t} = v_\pi(S_t) - R_{t+1} - \gamma v_\pi(S_{t+1}). \quad (49)$$

Bellman equation 保证 $\mathbb{E}_\pi[e_{\pi,t} | S_t = s] = 0$, 不是每次 $e_{\pi,t} = 0$ 。校订说明: 原稿 “TD error 非零说明 value 不正确” 须限定为非零的条件期望 Bellman residual; 单个非零 error 可能仅来自环境随机性。

7.3 Stochastic approximation 与 bootstrapping

真实 value 满足

$$v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s]. \quad (50)$$

若 $v_\pi(S_{t+1})$ 已知, 就可直接用样本作 Robbins–Monro 更新。实际以当前估计 $v_t(S_{t+1})$ 替代, 得到式 (47)。这种 “用估计更新估计” 称为 bootstrapping。向量 root 为 $g(v) = v - r_\pi - \gamma P_\pi v = 0$ 。

表格型 TD 的理论要求每个要学习的 state 反复被访问, 实际 step sizes 满足

$$\sum_t \alpha_t(s) = \infty, \quad \sum_t \alpha_t(s)^2 < \infty, \quad (51)$$

其中未访问 s 时记 $\alpha_t(s) = 0$ 。常数 step size 不满足第二个条件, 不应直接援引此 almost sure convergence 结论; 它可能在目标附近持续波动。

MC 通常等待 episode return, TD 每得到一个 transition 即可更新。原稿 “MC offline、TD online” 指更新时机, 不是说 MC 只能学习固定离线数据集。MC 与 TD 的 variance/bias 比较是完整随机 return 与 bootstrap target 的常见权衡, 不应理解为任意问题下的绝对排序。

8 SARSA、n-step SARSA 与 Q-learning

对应原笔记第 12–14 页。

8.1 SARSA 的五元组与 control

SARSA 使用 $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$, 下一步 action 按当前 policy 采样:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) + \alpha_t [r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1}) - q_t(s_t, a_t)]. \quad (52)$$

其他 pairs 不变。固定 policy 时是在估计 q_π ; control 则同时改进 ϵ -greedy policy。

- 1: 初始化 q , 令 π 为关于 q 的 ϵ -greedy policy。
- 2: **for** 每个 episode **do**
- 3: 初始化 s , 采样 $a \sim \pi(\cdot | s)$ 。
- 4: **while** s 不是 terminal state **do**
- 5: 执行 a , 观察 r, s' 。
- 6: **if** s' 是 terminal state **then**
- 7: $y \leftarrow r$ 。
- 8: **else**
- 9: 采样 $a' \sim \pi(\cdot | s')$, 令 $y \leftarrow r + \gamma q(s', a')$ 。
- 10: **end if**
- 11: $q(s, a) \leftarrow q(s, a) + \alpha[y - q(s, a)]$ 。
- 12: 在 s 处按更新后的 q 改进 ϵ -greedy policy。
- 13: $s \leftarrow s'$; 若未终止, 则 $a \leftarrow a'$ 。
- 14: **end while**
- 15: **end for**

8.2 n-step target: 连接 TD 与 MC

先用真实 q_π 定义

$$G_t^{(1)} = R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}), \quad (53)$$

$$G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 q_\pi(S_{t+2}, A_{t+2}), \quad (54)$$

$$G_t^{(n)} = \sum_{j=0}^{n-1} \gamma^j R_{t+j+1} + \gamma^n q_\pi(S_{t+n}, A_{t+n}). \quad (55)$$

这些 target 与完整 return 不是逐样本相等, 但给定 (S_t, A_t) 后、遵循 π 时, 其期望均为 $q_\pi(S_t, A_t)$ 。实际用当前 q 代替 q_π , 收集后续 n 步后更新:

$$Y_{t:t+n} = \sum_{j=0}^{n-1} \gamma^j r_{t+j+1} + \gamma^n q_{t+n-1}(s_{t+n}, a_{t+n}), \quad (56)$$

$$q_{t+n}(s_t, a_t) = q_{t+n-1}(s_t, a_t) + \alpha[Y_{t:t+n} - q_{t+n-1}(s_t, a_t)].$$

提前终止时累加到终点, 不再 bootstrap。 $n = 1$ 对应 SARSA, 一直等到终止则得到 MC target。较大 n 通常增大 return 随机性、减弱 bootstrap 依赖。

8.3 Q-learning 的 optimal action value target

将下一步采样 action 的 value 改为最大值:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) + \alpha_t \left[r_{t+1} + \gamma \max_a q_t(s_{t+1}, a) - q_t(s_t, a_t) \right]. \quad (57)$$

对应 Bellman optimality equation:

$$q^*(s, a) = r(s, a) + \gamma \sum_{s'} p(s' | s, a) \max_{a'} q^*(s', a'). \quad (58)$$

令 $v^*(s) = \max_a q^*(s, a)$, 便与 state-value BOE 联系起来。这里不必每轮先完整评估某个固定 policy 再改进。

8.4 On-policy 与 off-policy

Behavior policy 负责采样; target policy 是被评估或改进的 policy。

- SARSA 的 a_{t+1} 来自正在学习的 policy, 其基本形式是 on-policy。
- 基本 MC evaluation 的后续 trajectory 遵循待评估 policy; off-policy MC 需要额外的分布修正。
- Q-learning 可用探索性的 π_b 采样, target 却使用当前 q 的 greedy action, 因此是 off-policy。

给定 (s_t, a_t) 后 reward/transition 的条件分布由环境决定, 不等于整个 experience 分布与 behavior policy 无关。原稿 off-policy Q-learning 流程为: 用 π_b 采集 episodes, 逐 transition 更新 q , 令 target policy 选择 $\arg \max_a q(s, a)$ 。Target policy 无须承担 exploration, 但 behavior policy 需要覆盖要学习的 pairs。二者偶然一致, 也不改变 Q-learning 使用 greedy bootstrap target 的机制。

9 Value Function Approximation 与 Deep Q-learning

对应原笔记第 15–16 页。

9.1 用参数化函数近似 value

当 state 太多时，不再为每个 state 单独存表，而用 $\hat{v}(s, w)$ 近似 $v_\pi(s)$ 。若 S 服从 uniform distribution，原稿的均方误差目标为

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2] = \frac{1}{|\mathcal{S}|} \sum_s (v_\pi(s) - \hat{v}(s, w))^2. \quad (59)$$

也可按 policy 的长期访问频率加权。Stationary distribution d_π 满足

$$d_\pi(s) \geq 0, \quad \sum_s d_\pi(s) = 1, \quad d_\pi^\top P_\pi = d_\pi^\top. \quad (60)$$

于是

$$J(w) = \sum_s d_\pi(s) (v_\pi(s) - \hat{v}(s, w))^2. \quad (61)$$

Stationary distribution 描述 long-run behavior；唯一性及是否与初始 state 无关需要相应 Markov chain 条件，并非只写出上述方程就自动成立。

固定采样权重后，gradient 为

$$\nabla_w J(w) = -2\mathbb{E}[(v_\pi(S) - \hat{v}(S, w))\nabla_w \hat{v}(S, w)]. \quad (62)$$

把系数 2 吸收到 step size，SGD 形式为

$$w_{t+1} = w_t + \alpha_t [v_\pi(s_t) - \hat{v}(s_t, w_t)] \nabla_w \hat{v}(s_t, w_t). \quad (63)$$

真实 target 不可得时，可以替换为 episode return G_t ，或 $r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t)$ 。使用后者并把 target 当作常量求导，得到 semi-gradient TD。校订说明：原稿在写出负的 loss gradient 后，SGD 更新又用了负号；这里改为 prediction 向 target 靠近的正号形式。

9.2 Linear approximation 与表格型特例

若 $\hat{v}(s, w) = \phi(s)^\top w$ ，则

$$\nabla_w \hat{v}(s, w) = \phi(s), \quad w_{t+1} = w_t + \alpha_t [y_t - \phi(s_t)^\top w_t] \phi(s_t). \quad (64)$$

当 $\phi(s) = e_s$ 是 one-hot vector 时，每次只改变一个 state 对应的参数，退化为表格型 TD。

9.3 SARSA with function approximation

对 action value 使用 $\hat{q}(s, a, w)$ 。初始化 w_0, π_0 和 $\alpha > 0, \epsilon \in (0, 1)$ ；每个 episode 从 s_0 出发，目标是学习能到达 task 中目标状态的 policy。每一步执行当前 action，观察 reward 和下一 state，再从当前 policy 采样下一 action，更新

$$\begin{aligned} y_t &= r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t), \\ w_{t+1} &= w_t + \alpha [y_t - \hat{q}(s_t, a_t, w_t)] \nabla_w \hat{q}(s_t, a_t, w_t). \end{aligned} \quad (65)$$

根据新的 $\hat{q}$ 更新 ϵ -greedy policy，并令 $s_t \leftarrow s_{t+1}, a_t \leftarrow a_{t+1}$ ；终止后开始新 episode。校订说明：第 16 页顶部 ϵ -greedy 公式的非 greedy 分支写成 0，概率和不等于 1；应为式 (34) 中的 $\epsilon/|\mathcal{A}(s)|$ 。

若把 y_t 中的下一 action value 换成 $\max_a \hat{q}(s_{t+1}, a, w_t)$ ，就得到 Q-learning 型 semi-gradient target。即使 experience 由当前 ϵ -greedy policy 生成，它仍与 SARSA 的采样 action target 不同。

9.4 DQN 的 main network 与 target network

原稿引入两组参数：main network 参数 w ，以及在一段更新期间固定的 target network 参数 w_T 。对 transition (s, a, r, s') ，设

$$y_T = r + \gamma \max_{a' \in \mathcal{A}(s')} \hat{q}(s', a', w_T). \quad (66)$$

若 s' 真正终止，取 $y_T = r$ 。训练 main network 的平方误差为

$$J(w) = \mathbb{E}_{(S, A, R, S') \sim \mathcal{B}} [(y_T - \hat{q}(S, A, w))^2]. \quad (67)$$

其中 $\mathcal{B}$ 为 replay buffer 的采样分布。固定 w_T 后，

$$\nabla_w J(w) = -2\mathbb{E}[(y_T - \hat{q}(S, A, w)) \nabla_w \hat{q}(S, A, w)]. \quad (68)$$

原稿的网络示意图表示 $(s, a) \mapsto \hat{q}(s, a, w)$ ，本质上是以 (s, a, y_T) 为样本进行 regression；离散 action 实现也可让网络输入 s 并输出各个 action 的 values。

Experience replay 算法转写：

- 1: 初始化 main network w ，令 $w_T \leftarrow w$ 。
- 2: 用 behavior policy π_b 采集 transitions，存入 replay buffer $\mathcal{B}$ 。
- 3: **for** 每次训练 iteration **do**
- 4: 从 $\mathcal{B}$ 中 uniform sampling 一个 minibatch。
- 5: 对每个样本用固定的 w_T 计算 target y_T 。
- 6: 最小化 minibatch 中 $(y_T - \hat{q}(s, a, w))^2$ 的平均值，更新 w 。
- 7: **if** 达到每 C 次 iteration 的同步时刻 **then**
- 8: $w_T \leftarrow w$ 。
- 9: **end if**
- 10: **end for**

原稿把此流程作为基于已有 experience 的 off-policy 学习描述，不要求训练每一步都显式更新 behavior policy；学到 q 后可取 greedy policy。

校订说明： Bellman equation 在真实 q^* 处保证条件期望 TD residual 为零，不保证随机环境中每个平方 residual 都为零；因此 DQN 的上述平方 loss 不一定能降到零。原稿把 “behavior policy 随当前网络变化” 称为 on-policy 的批注也不准确：是否 off-policy 要比较采样 policy 和 bootstrap target，而非只看采样时是否使用当前网络。

第二部分 Policy Gradient 与 Actor-Critic

10 Policy Gradient 的 Performance Metrics

对应原笔记第 17-18 页。

10.1 Average value 与固定初始分布

为了直接优化 policy, 令 $\pi_\theta(a | s) = \pi(a | s, \theta)$ 。首先定义某个 state distribution d 下的 average value:

$$\bar{v}_\pi = \sum_s d(s) v_\pi(s) = d^\top v_\pi = \mathbb{E}_{S \sim d}[v_\pi(S)]. \quad (69)$$

原稿区分两种情形:

- d 与 policy 无关, 例如 uniform distribution, 或 $d(s_0) = 1$ 、其他 state 概率为零。
- $d = d_\pi$ 随 policy 变化, 取 policy 诱导的 stationary distribution。

为避免混淆, 以下用

$$J_0(\theta) = \sum_s d_0(s) v_{\pi_\theta}(s) \quad (70)$$

专门表示固定初始分布 d_0 的 discounted objective。

10.2 Average reward 与长期平均

记

$$\bar{r}_\pi = \sum_s d_\pi(s) r_\pi(s), \quad r_\pi(s) = \sum_a \pi(a | s) r(s, a). \quad (71)$$

在固定 policy 诱导的有限 Markov chain 满足适当 ergodicity 条件时, 长期 expected average reward 与初始 state 无关, 并等于 $\bar{r}_\pi$ 。原稿的证明可整理为

$$\frac{1}{N} \mathbb{E}_\pi \left[\sum_{k=1}^N R_{t+k} \mid S_t = s_0 \right] = \frac{1}{N} \sum_{k=1}^N \sum_s P_\pi^{k-1}(s \mid s_0) r_\pi(s) \quad (72)$$

$$= \sum_s \left[\frac{1}{N} \sum_{k=1}^N P_\pi^{k-1}(s \mid s_0) \right] r_\pi(s) \longrightarrow \bar{r}_\pi. \quad (73)$$

其中 $P_\pi^{k-1}(s \mid s_0)$ 表示从 s_0 出发经过 $k-1$ 次 transition 到达 s 的概率; 下标与 reward R_{t+k} 对齐。

若初始 state 已服从 d_π , 则 discounted return 的期望满足

$$\mathbb{E}_{S_0 \sim d_\pi, \pi} \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \right] = \sum_s d_\pi(s) v_\pi(s) = \bar{v}_\pi. \quad (74)$$

在此处 $\bar{v}_\pi$ 专指 stationary weighting。左乘 Bellman equation 以 $d_\pi^\top$ ，利用 stationarity,

$$\bar{v}_\pi = \bar{r}_\pi + \gamma d_\pi^\top P_\pi v_\pi = \bar{r}_\pi + \gamma \bar{v}_\pi, \quad \boxed{\bar{r}_\pi = (1 - \gamma) \bar{v}_\pi}. \quad (75)$$

因此固定 γ 时，这两个 stationary-weighted 指标的最大化等价；不能把这个等式直接套到任意固定 d_0 上。

11 Policy Gradient Theorem 与 REINFORCE

对应原笔记第 18–19 页。

11.1 固定初始分布的 gradient 推导

环境模型不依赖 θ 。从 $v_\pi(s) = \sum_a \pi_\theta(a | s) q_\pi(s, a)$ 出发，

$$\nabla_\theta v_\pi(s) = \sum_a \nabla_\theta \pi_\theta(a | s) q_\pi(s, a) + \gamma \sum_{s'} p_\pi(s' | s) \nabla_\theta v_\pi(s'). \quad (76)$$

记第一项为 $h_\theta(s)$ ，不断展开后续项，得到

$$\nabla_\theta v_\pi(s) = \sum_x \sum_{k=0}^{\infty} \gamma^k P_\pi^k(x | s) h_\theta(x). \quad (77)$$

定义 discounted visitation frequency

$$\rho_{\pi, d_0}(x) = \sum_s d_0(s) \sum_{k=0}^{\infty} \gamma^k P_\pi^k(x | s). \quad (78)$$

则固定初始分布下的 policy gradient 为

$$\boxed{\nabla_\theta J_0(\theta) = \sum_s \rho_{\pi, d_0}(s) \sum_a \nabla_\theta \pi_\theta(a | s) q_\pi(s, a)}. \quad (79)$$

在无限 discounted 设定下 $\sum_s \rho_{\pi, d_0}(s) = 1/(1 - \gamma)$ ，所以 ρ 本身不是归一化概率分布。令 $d_{\pi, d_0}^\gamma = (1 - \gamma) \rho_{\pi, d_0}$ ，才能按通常意义写成

$$\nabla_\theta J_0(\theta) = \frac{1}{1 - \gamma} \mathbb{E}_{S \sim d_{\pi, d_0}^\gamma, A \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(A | S) q_\pi(S, A)]. \quad (80)$$

这里使用 score-function identity:

$$\nabla_\theta \pi_\theta(a | s) = \pi_\theta(a | s) \nabla_\theta \log \pi_\theta(a | s). \quad (81)$$

离散 action 下可用 softmax 保证各 action 概率为正:

$$\pi_\theta(a | s) = \frac{\exp(h(s, a, \theta))}{\sum_{a'} \exp(h(s, a', \theta))}. \quad (82)$$

11.2 不同 objective 不能混用同一个权重

原稿将 gradient 统一概括为 $\sum_s \eta(s) \sum_a \nabla_{\theta} \pi_{\theta}(a | s) q_{\pi}(s, a)$ ，并明确提醒其中的等号有时表示近似或正比例， η 随 objective 改变。这里保留该提醒，并用式 (79) 固定 discounted 情况的精确定义。

校订说明：average reward 的精确 policy gradient 使用 stationary distribution 和相应的 differential action value，不能直接把 ρ_{π, d_0} 换成 d_{π} 而仍保留同一个 discounted q_{π} 。若用 q_{π}^{diff} 表示 differential action value，则相应形式为

$$\nabla_{\theta} \bar{r}_{\pi} = \sum_s d_{\pi}(s) \sum_a \nabla_{\theta} \pi_{\theta}(a | s) q_{\pi}^{\text{diff}}(s, a). \quad (83)$$

这里的 differential value 以扣除 $\bar{r}_{\pi}$ 后的长期 reward 为基础，满足相应的 Poisson/Bellman equation。本处指标区分可对照文末校订参考资料 1。

11.3 单次 policy update 的直观意义

用一个样本近似 gradient，可写成

$$\theta_{t+1} = \theta_t + \alpha \frac{\hat{q}_t(s_t, a_t)}{\pi_{\theta_t}(a_t | s_t)} \nabla_{\theta} \pi_{\theta_t}(a_t | s_t). \quad (84)$$

记 $\beta_t = \hat{q}_t(s_t, a_t) / \pi_{\theta_t}(a_t | s_t)$ ，在参数增量足够小时，一阶 Taylor 展开给出

$$\pi_{\theta_{t+1}}(a_t | s_t) \approx \pi_{\theta_t}(a_t | s_t) + \nabla_{\theta} \pi_{\theta_t}(a_t | s_t)^{\top} (\theta_{t+1} - \theta_t) \quad (85)$$

$$= \pi_{\theta_t}(a_t | s_t) + \alpha \beta_t \|\nabla_{\theta} \pi_{\theta_t}(a_t | s_t)\|^2. \quad (86)$$

若 $\beta_t > 0$ 且 gradient 非零，该 action 的概率在一阶近似下增大； $\beta_t < 0$ 时方向相反。原稿用“高 value action 得到加强”解释该式。**校订说明：**小概率出现在分母，并不单独保证有效 exploration；还要考虑采样概率、score gradient 和 value 的符号。

11.4 Monte Carlo Policy Gradient: REINFORCE

REINFORCE 用完整 episode 的 return 代替 q_{π} ：

$$G_t = \sum_{j=t}^{T-1} \gamma^{j-t} r_{j+1}. \quad (87)$$

原截图先生成 episode，再逐步计算 return 并更新 policy。下面整理为与固定初始分布 discounted objective 一致的版本：

- 1: 初始化 differentiable policy π_{θ} ，给定 γ 和 step size。
- 2: **for** $k = 0, 1, 2, \dots$ **do**
- 3: 采样 $s_0 \sim d_0$ ，用固定 θ_k 生成完整 episode。
- 4: 反向计算所有 $G_t = r_{t+1} + \gamma G_{t+1}$ ，终点 $G_T = 0$ 。
- 5: $g_k \leftarrow \sum_{t=0}^{T-1} \gamma^t G_t \nabla_{\theta} \log \pi_{\theta_k}(a_t | s_t)$ 。

6: $\theta_{k+1} \leftarrow \theta_k + \alpha_k g_k$ 。
 7: **end for**

校订说明：原截图的简化更新未显式写外层 γ^t ，且混用了 episode 与时间步参数下标。这里固定采样 policy 后累计 gradient 再更新； γ^t 对应 J_0 的时间折扣。若采用其他 state weighting 或目标，需要另行说明，不能声称不同采样约定下的简化式都是同一 objective 的无偏 gradient。原稿“offline、on-policy”的批注在这里表示等待 episode 完成后，使用本轮 policy 自己采集的数据。

12 Actor-Critic、Baseline 与 Advantage

对应原笔记第 19–21 页。

12.1 Actor 与 Critic 的分工：QAC

Actor 更新 policy 参数 θ ，Critic 进行 policy evaluation 或 value estimation。原稿把 MC return 和 TD value estimate 都视作给 actor 提供评价的信号；REINFORCE 本身并不一定包含一个单独训练的 critic network。

最基本的 Q Actor-Critic (QAC) 用 $\hat{q}(s, a, w)$ 作 critic。每一步由当前 policy 采样 a_t ，观察 r_{t+1}, s_{t+1} ，再在 s_{t+1} 采样 a_{t+1} 。原稿的局部更新为

$$\delta_t^q = r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t), \quad (88)$$

$$w_{t+1} = w_t + \alpha_w \delta_t^q \nabla_w \hat{q}(s_t, a_t, w_t), \quad (89)$$

$$\theta_{t+1} = \theta_t + \alpha_\theta \nabla_\theta \log \pi_{\theta_t}(a_t | s_t) \hat{q}(s_t, a_t, w_{t+1}). \quad (90)$$

Critic 使用 SARSA 型 TD target，actor 用其评价改进 policy。这是一种 on-policy 更新形式。Stochastic policy 本身可以采样探索，不必额外套用 ϵ -greedy；但这不代表无需关注探索质量。这些是单样本更新形式，整体 objective 的 state/time weighting 仍需与上一节对应，也不保证一般 neural network 一定找到全局最优 policy。

12.2 Action-independent baseline 不改变期望 gradient

令 $b(s)$ 不依赖 action；在 actor 的 score-function 项中把它当作固定评价量。对任意固定 state weighting η ，

$$\mathbb{E}_{S \sim \eta, A \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(A | S) b(S)] = \sum_s \eta(s) b(s) \sum_a \nabla_\theta \pi_\theta(a | s) \quad (91)$$

$$= \sum_s \eta(s) b(s) \nabla_\theta 1 = 0. \quad (92)$$

因此，用 $q_\pi(s, a) - b(s)$ 代替 $q_\pi(s, a)$ ，不会改变这部分 gradient 的期望，但会改变 variance。

12.3 最小化 gradient estimator 的 variance

记

$$z(S, A) = \nabla_{\theta} \log \pi_{\theta}(A | S), \quad X = z(S, A)[q_{\pi}(S, A) - b(S)], \quad \bar{x} = \mathbb{E}[X]. \quad (93)$$

当 $\theta \in \mathbb{R}^d$ 时, $X \in \mathbb{R}^d$, 其 covariance matrix 为

$$\text{Cov}(X) = \mathbb{E}[(X - \bar{x})(X - \bar{x})^{\top}] = \mathbb{E}[XX^{\top}] - \bar{x}\bar{x}^{\top}, \quad (94)$$

$$\text{tr Cov}(X) = \mathbb{E}[X^{\top}X] - \bar{x}^{\top}\bar{x}. \quad (95)$$

由于 $\bar{x}$ 不随 baseline 改变, 最小化 covariance 的 trace 等价于最小化

$$\mathbb{E}[X^{\top}X] = \mathbb{E}[\|z(S, A)\|^2 (q_{\pi}(S, A) - b(S))^2]. \quad (96)$$

对每个有正权重的 state s , 可分别最小化条件期望。记 $b = b(s)$,

$$F_s(b) = \mathbb{E}_{A \sim \pi(\cdot | s)}[\|z(s, A)\|^2 (q_{\pi}(s, A) - b)^2] \quad (97)$$

$$= b^2 \mathbb{E}[\|z\|^2 | s] - 2b \mathbb{E}[\|z\|^2 q_{\pi} | s] + \mathbb{E}[\|z\|^2 q_{\pi}^2 | s]. \quad (98)$$

若分母非零, 令 $F'_s(b) = 0$ 得

$$b^*(s) = \frac{\mathbb{E}_{A \sim \pi(\cdot | s)}[\|z(s, A)\|^2 q_{\pi}(s, A)]}{\mathbb{E}_{A \sim \pi(\cdot | s)}[\|z(s, A)\|^2]}. \quad (99)$$

若分母为零, 该 state 的 score gradient 几乎处处为零, baseline 选择不影响这一项。原稿的“最小 variance”在向量情况下具体指上述 trace 准则。

12.4 用 state value 作 baseline

忽略最优 baseline 中的 score-norm 加权, 一个自然的简化选择是

$$b(s) = \mathbb{E}_{A \sim \pi(\cdot | s)}[q_{\pi}(s, A)] = v_{\pi}(s). \quad (100)$$

于是出现 advantage function:

$$A_{\pi}(s, a) = q_{\pi}(s, a) - v_{\pi}(s). \quad (101)$$

它衡量该 action 相对于该 state 下平均表现的好坏。原稿将此 baseline 标为简化的、一般不保证最小 variance 的选择。Actor 的单样本更新为

$$\theta_{t+1} = \theta_t + \alpha_{\theta} \hat{A}_t \nabla_{\theta} \log \pi_{\theta_t}(a_t | s_t). \quad (102)$$

当采用真实 v_{π} 时,

$$\mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) - v_{\pi}(S_t) | S_t = s, A_t = a] = A_{\pi}(s, a). \quad (103)$$

因此, 一步 TD error 可以作为 advantage 的样本估计; 用近似 critic 时还会带有 value estimation error。

12.5 Advantage Actor–Critic / TD Actor–Critic

原笔记在此使用 A2C 标题。其单步算法转写为

- 1: 初始化 actor π_θ 和 state-value critic $\hat{v}(s, w)$ 。
- 2: **for** 每个 episode 的每一步 **do**
- 3: 采样 $a_t \sim \pi_{\theta_t}(\cdot | s_t)$, 观察 r_{t+1}, s_{t+1} 。
- 4: $\delta_t \leftarrow r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)$ 。
- 5: 若真正终止, 上一行的下一 state value 取 0。
- 6: Critic: $w_{t+1} \leftarrow w_t + \alpha_w \delta_t \nabla_w \hat{v}(s_t, w_t)$ 。
- 7: Actor: $\theta_{t+1} \leftarrow \theta_t + \alpha_\theta \delta_t \nabla_\theta \log \pi_{\theta_t}(a_t | s_t)$ 。
- 8: **end for**

这里的 δ_t 是 target 减 prediction, 与原稿前面 TD 部分的 e_t 符号相反。Actor 更新时不穿过 δ_t 对 critic 反向传播; critic 采用 semi-gradient target。原稿旁注的 π/β importance weight 属于下一节的 off-policy 扩展, 不应混入这段基本 on-policy 算法。

13 Importance Sampling 与 Off-policy Actor–Critic

对应原笔记第 21–22 页。

13.1 Inverse transform sampling

对 cumulative distribution function F , 若 $U \sim \text{Uniform}[0, 1]$, 令 $X = F^{-1}(U)$, 则

$$\Pr(X \leq x) = \Pr(F^{-1}(U) \leq x) = \Pr(U \leq F(x)) = F(x). \quad (104)$$

一般分布中 F^{-1} 取 generalized inverse。原稿指出: 这种方法直观, 但 F^{-1} 未必容易显式求出。

13.2 换分布计算期望

令 p 是 target distribution, b 是易于采样的 proposal/behavior distribution。只要 target 的有效 support 被 b 覆盖,

$$\mathbb{E}_{X \sim p}[f(X)] = \sum_x b(x) \frac{p(x)}{b(x)} f(x) = \mathbb{E}_{X \sim b} \left[\frac{p(X)}{b(X)} f(X) \right]. \quad (105)$$

连续情形将求和替换为积分, ratio 使用 density。原稿先以 $f(X) = X$ 说明; 其“容易/难以采样”的 p_0, p_1 文字标记与后续等式方向不一致, 这里用 target/behavior 区分, 明确分母是实际采样分布。

13.3 Actor 的 action-level importance weight

若 state 来自固定 weighting d_β , action 来自 behavior policy β , 则对同一个 state 有

$$\omega_\theta(s, a) = \frac{\pi_\theta(a | s)}{\beta(a | s)}. \quad (106)$$

由 importance sampling, 下面这个局部 actor 方向可改写为

$$g_{\text{local}}(\theta) = \sum_s d_\beta(s) \sum_a \nabla_\theta \pi_\theta(a | s) q_\pi(s, a) \quad (107)$$

$$= \mathbb{E}_{S \sim d_\beta, A \sim \beta} [\omega_\theta(S, A) \nabla_\theta \log \pi_\theta(A | S) q_\pi(S, A)]. \quad (108)$$

可将 q_π 换成 advantage 评价量。在 state-value critic 中, 单步 TD 更新也可在相应 off-policy 设定下引入该 action ratio, 但这不是一般函数近似收敛性的保证。

校订说明: 原稿把上述局部方向直接写成 $J_\beta(\theta) = \sum_s d_\beta(s) v_\pi(s)$ 的精确 gradient, 遗漏了未来状态的影响。由本笔记已推导的式 (79), 固定 d_β 时的精确形式应为

$$\nabla_\theta J_\beta = \sum_s \rho_{\pi, d_\beta}(s) \sum_a \nabla_\theta \pi_\theta(a | s) q_\pi(s, a), \quad \rho_{\pi, d_\beta}^\top = d_\beta^\top (I - \gamma P_\pi)^{-1}. \quad (109)$$

Action ratio 修正 action distribution, 不会自动把 d_β 修正成 ρ_{π, d_β} ; 使用近似方向时应明确其近似性质。

14 Deterministic Policy Gradient

对应原笔记第 22 页。

在 continuous action space 中, deterministic policy 直接输出 $a = \mu_\theta(s)$ 。对固定初始分布的 discounted objective, 在适当可微性等条件下, Deterministic Policy Gradient (DPG) 为

$$\nabla_\theta J_0(\theta) = \sum_s \rho_{\mu, d_0}(s) [D_\theta \mu_\theta(s)]^\top \nabla_a q_\mu(s, a)|_{a=\mu_\theta(s)}. \quad (110)$$

这里 $D_\theta \mu_\theta$ 是 action 对参数的 Jacobian; 连续 state 时求和改为积分。若以归一化 d_{μ, d_0}^γ 写 expectation, 应带上 $1/(1 - \gamma)$ 系数。

原稿强调, 与 stochastic policy gradient 不同, 这个公式不需要对 action distribution 再求期望: actor 给出 action, critic 对 action 求导, 再通过 chain rule 传回 actor 参数。它可以配合带探索的 behavior policy 构造 off-policy Actor-Critic, 但原始定理中的 state visitation weighting 仍与 target policy 有关, 不能任意替换而宣称 gradient 完全相同。该区分可对照文末资料 2。

原笔记最后列出 DPG、DDPG 作为延伸阅读关键词, 没有继续写出 DDPG 的完整算法; 此处保留该线索, 不把额外内容伪装成原稿转写。

第三部分 TRPO: 理论与优化

15 TRPO 的做法

TRPO 使用带有 KL divergence 约束的优化目标:

$$\begin{aligned} & \underset{\theta}{\text{maximize}} \quad \widehat{\mathbb{E}}_t \left[\frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \widehat{A}_t \right], \\ & \text{subject to} \quad \widehat{\mathbb{E}}_t [\text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_\theta(\cdot | s_t)]] \leq \delta. \end{aligned} \quad (111)$$

课件从 Lagrangian duality 与 KKT conditions 引出对应的 penalty 形式:

$$\underset{\theta}{\text{maximize}} \quad \widehat{\mathbb{E}}_t \left[\frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \widehat{A}_t - \beta \text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_\theta(\cdot | s_t)] \right]. \quad (112)$$

- TRPO 不采用上述 penalty 形式, 而是使用 hard constraint。
- 原课件概括其实现代价为: 复杂、速度慢、开销高。

16 TRPO 与 Performance Difference Lemma

对应原笔记第 23 页。

Trust Region Policy Optimization (TRPO) 用 KL divergence 衡量新旧 policy 的差异, 通过限制更新范围来控制性能变化。以下先保留原笔记的理论推导, 再区分实际近似。设固定初始分布为 d_0 , performance objective 为

$$\eta(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \right] = \mathbb{E}_{S_0 \sim d_0} [v_\pi(S_0)]. \quad (113)$$

原稿有时把 reward 简写成 $r(s_t)$; 这里沿用前文允许依赖 action 的一般记法。

16.1 Performance difference 的望远镜求和证明

令 $\tilde{\pi}$ 为新 policy。Performance Difference Lemma 表示

$$\boxed{\eta(\tilde{\pi}) - \eta(\pi) = \mathbb{E}_{\tau \sim \tilde{\pi}} \left[\sum_{t=0}^{\infty} \gamma^t A_\pi(S_t, A_t) \right]}. \quad (114)$$

由 Bellman expectation equation,

$$A_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) - v_\pi(s) | S_t = s, A_t = a]. \quad (115)$$

沿新 policy 的 trajectory 取期望, 乘以 γ^t 并对 t 求和。Value 项发生 telescope cancellation:

$$\sum_{t=0}^{T-1} (\gamma^{t+1} v_\pi(S_{t+1}) - \gamma^t v_\pi(S_t)) = \gamma^T v_\pi(S_T) - v_\pi(S_0), \quad (116)$$

$$\lim_{T \rightarrow \infty} \gamma^T v_\pi(S_T) = 0. \quad (117)$$

由于 value 有界, 后式成立。剩下的 reward 项期望为 $\eta(\tilde{\pi})$, 初始 value 项期望为 $\eta(\pi)$, 即得式 (114)。

16.2 用 discounted visitation frequency 重写

简记 $\rho_\pi = \rho_{\pi, d_0}$, 则

$$\rho_{\tilde{\pi}}(s) = \sum_{t=0}^{\infty} \gamma^t \Pr_{\tilde{\pi}}(S_t = s). \quad (118)$$

每个时刻都有

$$\mathbb{E}_{\tilde{\pi}}[A_\pi(S_t, A_t)] = \sum_s \sum_a \Pr_{\tilde{\pi}}(S_t = s) \tilde{\pi}(a | s) A_\pi(s, a). \quad (119)$$

交换求和次序后, performance difference 变为

$$\eta(\tilde{\pi}) = \eta(\pi) + \sum_s \rho_{\tilde{\pi}}(s) \sum_a \tilde{\pi}(a | s) A_\pi(s, a). \quad (120)$$

新 policy 同时影响 action distribution 和 state visitation distribution, 这是构造 surrogate objective 时需要处理的关键。

17 Surrogate Objective、TV Distance 与 Pinsker Inequality

对应原笔记第 24 页及第 25 页顶部。

17.1 固定旧 policy 的 visitation distribution

把 $\rho_{\tilde{\pi}}$ 替换为 ρ_π , 定义

$$L_\pi(\tilde{\pi}) = \eta(\pi) + \sum_s \rho_\pi(s) \sum_a \tilde{\pi}(a | s) A_\pi(s, a). \quad (121)$$

在旧 policy 处, $L_\pi(\pi) = \eta(\pi)$, 且二者的一阶 gradient 相同。令 $F_s(\theta) = \sum_a \pi_\theta(a | s) A_{\pi_{\text{old}}}(s, a)$, 有

$$F_s(\theta_{\text{old}}) = 0. \quad (122)$$

Product rule 给出

$$\nabla_\theta(\rho_{\pi_\theta}(s) F_s(\theta)) = \nabla_\theta \rho_{\pi_\theta}(s) F_s(\theta) + \rho_{\pi_\theta}(s) \nabla_\theta F_s(\theta). \quad (123)$$

在 θ_{old} 处第一项消失, 故

$$\nabla_\theta \eta(\pi_\theta)|_{\theta_{\text{old}}} = \nabla_\theta L_{\pi_{\text{old}}}(\pi_\theta)|_{\theta_{\text{old}}}. \quad (124)$$

17.2 Performance lower bound

为避免与 ϵ -greedy 参数混淆, 令

$$B_A = \max_{s,a} |A_{\pi_{\text{old}}}(s, a)|, \quad \alpha_{\text{TV}} = \max_s D_{\text{TV}}(\pi_{\text{old}}(\cdot | s), \pi_{\text{new}}(\cdot | s)). \quad (125)$$

原稿记录的 theoretical bound 为

$$\eta(\pi_{\text{new}}) \geq L_{\pi_{\text{old}}}(\pi_{\text{new}}) - \frac{4\gamma B_A}{(1-\gamma)^2} \alpha_{\text{TV}}^2. \quad (126)$$

结合以下 Pinsker inequality, 可写成

$$\eta(\pi_{\text{new}}) \geq L_{\pi_{\text{old}}}(\pi_{\text{new}}) - \frac{2\gamma B_A}{(1-\gamma)^2} D_{\text{KL}}^{\max}(\pi_{\text{old}}, \pi_{\text{new}}), \quad (127)$$

其中 $D_{\text{KL}}^{\max}$ 对所有 state 的 KL 取最大值, $\log$ 为自然对数。本段保留原笔记的常数约定; 相关理论与实际近似的区别见资料 3。

17.3 Total variation 的两种写法

对离散分布 $P = (p_i), Q = (q_i)$,

$$D_{\text{TV}}(P, Q) = \frac{1}{2} \sum_i |p_i - q_i| = \sup_E |P(E) - Q(E)|. \quad (128)$$

令 $d_i = p_i - q_i$, 则 $\sum_i d_i = 0$ 。取 $E = \{i : d_i \geq 0\}$, 有

$$\sum_i |p_i - q_i| = \sum_{i \in E} (p_i - q_i) + \sum_{i \notin E} (q_i - p_i) = 2 \sum_{i \in E} (p_i - q_i). \quad (129)$$

因此取 $p = P(E), q = Q(E)$ 后, $p - q = D_{\text{TV}}(P, Q)$ 。

17.4 Log-sum inequality 与 Pinsker 的证明

KL divergence 为

$$D_{\text{KL}}(P \| Q) = \sum_i p_i \log \frac{p_i}{q_i}. \quad (130)$$

先证明 log-sum inequality:

$$\sum_i a_i \log \frac{a_i}{b_i} \geq \left(\sum_i a_i \right) \log \frac{\sum_i a_i}{\sum_i b_i}. \quad (131)$$

取 convex function $f(x) = x \log x$, 以 $b_i / \sum_j b_j$ 为权重, Jensen inequality 给出

$$\sum_i \frac{b_i}{\sum_j b_j} f\left(\frac{a_i}{b_i}\right) \geq f\left(\frac{\sum_i a_i}{\sum_i b_i}\right). \quad (132)$$

两边乘 $\sum_j b_j$ 即得。分别对 E 与补集应用,

$$D_{\text{KL}}(P \| Q) \geq p \log \frac{p}{q} + (1-p) \log \frac{1-p}{1-q} = D_{\text{KL}}(\text{Bern}(p) \| \text{Bern}(q)). \quad (133)$$

固定 $q \in (0, 1)$, 记右侧为 $f_q(p)$, 则

$$f_q(q) = 0, \quad f'_q(q) = 0, \quad f''_q(p) = \frac{1}{p} + \frac{1}{1-p} = \frac{1}{p(1-p)} \geq 4. \quad (134)$$

由二阶 Taylor 余项或对二阶导数下界积分,

$$f_q(p) \geq 2(p-q)^2 = 2D_{\text{TV}}(P, Q)^2. \quad (135)$$

从而

$$\boxed{D_{\text{TV}}(P, Q) \leq \sqrt{\frac{1}{2} D_{\text{KL}}(P \| Q)}}. \quad (136)$$

边界情形按连续极限或 KL 为无穷处理。**校订说明:** 原稿最后一行漏掉 TV 的平方和系数, 应为 $2(p-q)^2 = 2D_{\text{TV}}^2$, 而不是等于 D_{TV} 。

18 TRPO 的约束近似与 Importance Ratio

对应原笔记第 25 页。

18.1 从理论 penalty 到实际 trust-region constraint

记 $C = 2\gamma B_A / (1 - \gamma)^2$, 由式 (127) 定义

$$M(\tilde{\pi}) = L_{\pi}(\tilde{\pi}) - CD_{\text{KL}}^{\max}(\pi, \tilde{\pi}). \quad (137)$$

有 $\eta(\tilde{\pi}) \geq M(\tilde{\pi})$ 且 $\eta(\pi) = M(\pi)$ 。因此若能保证 $M(\tilde{\pi}) \geq M(\pi)$, 就能保证相应 theoretical performance 不下降。原稿指出, C 较大时 penalty 很保守, 参数改动会很小。

实际采用 surrogate maximization 加 KL constraint。最大 KL 要求所有 state 的差异都受控, 难以直接实现; 平均 KL 则只约束旧 policy 访问分布下的平均差异。用归一化 d_{old}^{γ} 定义

$$\overline{D}_{\text{KL}}(\theta_{\text{old}}, \theta) = \mathbb{E}_{S \sim d_{\text{old}}^{\gamma}} [D_{\text{KL}}(\pi_{\text{old}}(\cdot | S) \| \pi_{\theta}(\cdot | S))]. \quad (138)$$

采样实现还会用 rollout 的经验 state distribution 近似该期望。**校订说明:** 平均 KL、有限样本和后续 Taylor approximation 都属于 practical approximation, 不能直接继承最大 KL 理论界的全部保证。

18.2 用旧 action 样本评估新 policy

忽略与 θ 无关的加法常数和固定正比例系数后, surrogate 为

$$\mathcal{L}_{\text{old}}(\theta) = \mathbb{E}_{S \sim d_{\text{old}}^{\gamma}, A \sim \pi_{\theta}} [A_{\pi_{\text{old}}}(S, A)]. \quad (139)$$

但数据中的 action 来自旧 policy。由

$$\mathbb{E}_{A \sim \pi_{\theta}} [f(A)] = \mathbb{E}_{A \sim \pi_{\text{old}}} \left[\frac{\pi_{\theta}(A | s)}{\pi_{\text{old}}(A | s)} f(A) \right], \quad (140)$$

得到

$$\mathcal{L}_{\text{old}}(\theta) = \mathbb{E}_{S \sim d_{\text{old}}^{\gamma}, A \sim \pi_{\text{old}}} \left[\frac{\pi_{\theta}(A | S)}{\pi_{\text{old}}(A | S)} A_{\pi_{\text{old}}}(S, A) \right]. \quad (141)$$

实现时使用保存的 old log probability 和 advantage estimate。原截图的代码转写为

```
log_probs = policy_net.get_log_prob(states, actions)
ratio = torch.exp(log_probs - old_log_probs)
surrogate = (ratio * advantages).mean()
```

其依据是 $\exp(\log \pi_{\theta} - \log \pi_{\text{old}}) = \pi_{\theta} / \pi_{\text{old}}$ 。原稿“先 log 再 exp 防止 NaN/Inf”应理解为改善数值计算的常用形式，不是绝对防溢出保证。这里的 surrogate 要最大化；若交给默认最小化的 optimizer，应使用负号。

18.3 一阶 objective 与二阶 KL

令 $\Delta\theta = \theta - \theta_{\text{old}}$, $g = \nabla_{\theta} \mathcal{L}_{\text{old}}(\theta_{\text{old}})$, 则

$$\mathcal{L}_{\text{old}}(\theta) \approx \mathcal{L}_{\text{old}}(\theta_{\text{old}}) + g^{\top} \Delta\theta. \quad (142)$$

在旧参数处，平均 KL 为零，且在通常光滑条件下 gradient 为零。因此第一个反映分布变化的非零项为二阶项：

$$\bar{D}_{\text{KL}}(\theta_{\text{old}}, \theta) \approx \frac{1}{2} \Delta\theta^{\top} H \Delta\theta, \quad H = \nabla_{\theta}^2 \bar{D}_{\text{KL}}(\theta_{\text{old}}, \theta) \big|_{\theta=\theta_{\text{old}}}. \quad (143)$$

TRPO 的局部近似问题成为

$$\underset{\Delta\theta}{\text{maximize}} \quad g^{\top} \Delta\theta, \quad \text{subject to} \quad \frac{1}{2} \Delta\theta^{\top} H \Delta\theta \leq \delta. \quad (144)$$

这是一阶 objective 加二次 constraint 的优化问题，而不是对原非线性问题的精确改写。

19 TRPO 更新方向、Conjugate Gradient 与 Hessian–Vector Product

对应原笔记第 26 页；第 27 页为空白。

19.1 从 Lagrangian 得到局部最优步长

原截图先计算 surrogate 的参数 gradient，再展平成一个向量：

```
surrogate_grads = torch.autograd.grad(
    surrogate, policy_net.parameters()
)
flat_gradient = torch.cat(
    [gradient.reshape(-1) for gradient in surrogate_grads]
).detach()
```

对应数学中的 g 。对式 (144), Lagrangian 为

$$\mathcal{L}_{\text{lag}}(\Delta\theta, \lambda) = g^\top \Delta\theta - \lambda \left(\frac{1}{2} \Delta\theta^\top H \Delta\theta - \delta \right). \quad (145)$$

假设 H 正定、 $g \neq 0$ 、 $\delta > 0$ 。Stationarity 条件给出

$$g - \lambda H \Delta\theta = 0, \quad \Delta\theta = \frac{1}{\lambda} H^{-1} g. \quad (146)$$

代回 constraint,

$$\frac{1}{2\lambda^2} g^\top H^{-1} g \leq \delta, \quad \lambda \geq \sqrt{\frac{g^\top H^{-1} g}{2\delta}}. \quad (147)$$

为了最大化 $g^\top \Delta\theta$, 在约束边界取等号:

$$\lambda^* = \sqrt{\frac{g^\top H^{-1} g}{2\delta}}, \quad \Delta\theta^* = \sqrt{\frac{2\delta}{g^\top H^{-1} g}} H^{-1} g. \quad (148)$$

这是原稿推导的局部更新方向与缩放。若 $g = 0$, 可取零更新; Hessian 可能只有半正定性, 实际求解不能无条件求逆。

19.2 不显式求逆: Conjugate Gradient

令 $x = H^{-1}g$, 等价于求解线性系统 $Hx = g$ 。原稿提出使用 Conjugate Gradient (CG, 共轭梯度法), 避免显式构造逆矩阵。CG 所需的关键运算是 Hv , 不必保存整个 H 。对局部二次近似得到的步长, 实际 TRPO 还需检验真实 surrogate 与 KL; 原笔记未继续写出 backtracking line search 的算法。本处仅说明该推导的适用范围, 不把补充流程当作原稿已有内容。

19.3 Hessian-vector product 的证明

令 $f(\theta)$ 为二次可微 scalar function, $g_f(\theta) = \nabla_\theta f(\theta)$, $H_f(\theta) = \nabla_\theta^2 f(\theta)$ 。对与 θ 无关的向量 v ,

$$v^\top g_f(\theta) = \sum_i v_i \frac{\partial f}{\partial \theta_i}. \quad (149)$$

再对第 k 个参数求导:

$$\frac{\partial}{\partial \theta_k} (v^\top g_f(\theta)) = \sum_i v_i \frac{\partial^2 f}{\partial \theta_k \partial \theta_i} = [H_f(\theta)v]_k. \quad (150)$$

因此

$$H_f(\theta)v = \nabla_\theta \left(v^\top \nabla_\theta f(\theta) \right). \quad (151)$$

先求一次 gradient, 再与固定向量内积, 最后对所得 scalar 求 gradient, 就得到 Hessian-vector product。这也解释了为何可通过自动微分实现 TRPO 中的 CG 所需运算。

第四部分 PPO: 目标函数与梯度

Clipped Surrogate Objective function

$$L^{\text{CLIP}}(\theta) = \widehat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \widehat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \widehat{A}_t \right) \right]. \quad (152)$$

20 PPO 的改进: 从 hard constraint 到 soft constraint

原课件引用评论（雪碧孙尚香）：“有没有简单点的打法 [星星眼][星星眼]”

回应：“有的兄弟，有的！”

- 既然 TRPO 的 hard constraint 不便于实现，我们可不可以用 soft constraint？
- 不等式约束一定要严格遵守吗？偶尔违反几次会不会影响不大？
- 既然 β 不好选择，就使用动态的 β 。

由此得到 KL penalty objective:

$$L^{\text{KLPEN}}(\theta) = \widehat{\mathbb{E}}_t \left[\frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \widehat{A}_t - \beta \text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_\theta(\cdot | s_t)] \right]. \quad (153)$$

21 PPO with Adaptive KL Penalty

对于式 (153), 先确定一个目标 KL divergence d_{targ} , 再通过新旧 policy 的 KL divergence 动态调整 β :

- 若 KL divergence 过小, 减小 β 。
- 若 KL divergence 过大, 增大 β 。

计算

$$d = \hat{\mathbb{E}}_t[\text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)]], \quad (154)$$

并按课件中的规则更新:

$$\beta \leftarrow \begin{cases} \beta/2, & d < d_{\text{targ}}/1.5, \\ 2\beta, & d > 1.5 d_{\text{targ}}, \\ \beta, & \text{otherwise.} \end{cases} \quad (155)$$

Algorithm 4: PPO with Adaptive KL Penalty

Input: initial policy parameters θ_0 , initial KL penalty β_0 , target KL-divergence δ .

```

1: for  $k = 0, 1, 2, \dots$  do
2:   Collect partial trajectories  $\mathcal{D}_k$  on policy  $\pi_k = \pi(\theta_k)$ .
3:   Estimate advantages  $\hat{A}_t^{\pi_k}$  using an advantage estimation algorithm.
4:   Compute the policy update by taking  $K$  steps of minibatch SGD (via Adam):
        $\theta_{k+1} = \arg \max_{\theta} [\mathcal{L}_{\theta_k}(\theta) - \beta_k \overline{\text{D}}_{\text{KL}}(\theta \| \theta_k)]$ .
5:   if  $\overline{\text{D}}_{\text{KL}}(\theta_{k+1} \| \theta_k) \geq 1.5\delta$  then
6:      $\beta_{k+1} = 2\beta_k$ .
7:   else if  $\overline{\text{D}}_{\text{KL}}(\theta_{k+1} \| \theta_k) \leq \delta/1.5$  then
8:      $\beta_{k+1} = \beta_k/2$ .
9:   else
10:     $\beta_{k+1} = \beta_k$ .
11:   end if
12: end for

```

这里保留原课件算法的 $\overline{\text{D}}_{\text{KL}}(\theta \| \theta_k)$ 记号。原算法截图未写出中间区间的分支; 转写时将保持 β 不变的分支显式补全。目标值 δ 对应上文的 d_{targ} 。

22 PPO-clip

- PPO-penalty 随着训练过程调整 β 。
- PPO-clip 的课件直观描述是“直接把策略的改动限制在一个范围里”：具体操作是在 surrogate objective 中对 probability ratio 进行 clipping。
- PPO-penalty 用 KL divergence 衡量新旧 policy 的差异。
- PPO-clip 使用 importance weight，即 probability ratio：

$$p_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}. \quad (156)$$

22.1 CLIP function

$$\text{clip}(x, l, r) = \max(\min(x, r), l). \quad (157)$$

因此，

$$\text{clip}(p_t(\theta), 1 - \epsilon, 1 + \epsilon) = \begin{cases} 1 - \epsilon, & p_t(\theta) < 1 - \epsilon, \\ 1 + \epsilon, & p_t(\theta) > 1 + \epsilon, \\ p_t(\theta), & \text{otherwise.} \end{cases} \quad (158)$$

22.2 Clipped Surrogate Objective function

$$\begin{aligned} L_t^{\text{CLIP}}(\theta) &= \min(p_t(\theta)\hat{A}_t, \text{clip}(p_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t), \\ L^{\text{CLIP}}(\theta) &= \mathbb{E}_t[L_t^{\text{CLIP}}(\theta)]. \end{aligned} \quad (159)$$

22.3 min 和 CLIP 的梯度

原课件采用如下分支规则：

$$\frac{\partial \min(x, y)}{\partial x} = \begin{cases} 1, & x \leq y, \\ 0, & \text{otherwise,} \end{cases} \quad \frac{\partial \min(x, y)}{\partial y} = \begin{cases} 1, & y < x, \\ 0, & \text{otherwise,} \end{cases} \quad (160)$$

$$\frac{\partial \text{clip}(x, a, b)}{\partial x} = \begin{cases} 1, & a \leq x \leq b, \\ 0, & \text{otherwise.} \end{cases} \quad (161)$$

边界约定：原课件提醒，这些导数在数学上可能并不严谨，是其讲解所采用的 backward 分支约定。这里保留该约定，不将 $x = y$ 或 clipping 边界处的取值视为唯一的普通导数。

23 PPO 里梯度的传播

- 我们的目标是最大化 $L_t^{\text{CLIP}}(\theta)$ 。
- 主流 deep learning optimizer 使用最小化的目标形式。
- 因此取负，对 $-L_t^{\text{CLIP}}(\theta)$ 求梯度。

为便于排版，记

$$\begin{aligned} c_t(\theta) &= \text{clip}(p_t(\theta), 1 - \epsilon, 1 + \epsilon), \\ u_t &= p_t(\theta)\hat{A}_t, \quad v_t = c_t(\theta)\hat{A}_t, \\ L_t^{\text{CLIP}}(\theta) &= \min(u_t, v_t). \end{aligned} \tag{162}$$

将 $\hat{A}_t$ 和 old policy 视为固定量，课件中的 chain rule 可以写成

$$\frac{\partial(-L_t^{\text{CLIP}})}{\partial\pi_\theta(a_t | s_t)} = \frac{\partial(-L_t^{\text{CLIP}})}{\partial L_t^{\text{CLIP}}} \left(\frac{\partial L_t^{\text{CLIP}}}{\partial u_t} \frac{\partial u_t}{\partial p_t} + \frac{\partial L_t^{\text{CLIP}}}{\partial v_t} \frac{\partial v_t}{\partial c_t} \frac{\partial c_t}{\partial p_t} \right) \frac{\partial p_t}{\partial\pi_\theta(a_t | s_t)}. \tag{163}$$

定义与上一节边界约定一致的分支系数：

$$g_{u,t} = \begin{cases} 1, & u_t \leq v_t, \\ 0, & \text{otherwise}, \end{cases} \quad g_{v,t} = \begin{cases} 1, & v_t < u_t, \\ 0, & \text{otherwise}, \end{cases} \tag{164}$$

$$g_{c,t} = \begin{cases} 1, & 1 - \epsilon \leq p_t(\theta) \leq 1 + \epsilon, \\ 0, & \text{otherwise}. \end{cases} \tag{165}$$

代入

$$\frac{\partial(-L_t^{\text{CLIP}})}{\partial L_t^{\text{CLIP}}} = -1, \quad \frac{\partial u_t}{\partial p_t} = \frac{\partial v_t}{\partial c_t} = \hat{A}_t, \quad \frac{\partial p_t}{\partial\pi_\theta(a_t | s_t)} = \frac{1}{\pi_{\theta_{\text{old}}}(a_t | s_t)}, \tag{166}$$

得到原课件长公式的等价分支形式：

$$\boxed{\frac{\partial(-L_t^{\text{CLIP}})}{\partial\pi_\theta(a_t | s_t)} = - \left(g_{u,t}\hat{A}_t + g_{v,t}\hat{A}_t g_{c,t} \right) \frac{1}{\pi_{\theta_{\text{old}}}(a_t | s_t)}}. \tag{167}$$

原课件此页使用 A_t ，此处统一记为 $\hat{A}_t$ ，与 objective 中的 advantage estimate 对齐。

24 直观看每段的梯度

以下保留原课件的六种情况。假设 $p_t(\theta) > 0$ ，表中的 Gradient 表示该样本的 clipped surrogate 分支是否保留梯度，不表示整个训练 loss 的所有项都为零或非零。

$p_t(\theta)$	$\hat{A}_t$	Return value of min	Objective is clipped	Sign of objective	Gradient
1 $[1 - \epsilon, 1 + \epsilon]$	+	$p_t(\theta)\hat{A}_t$	no	+	✓
2 $[1 - \epsilon, 1 + \epsilon]$	-	$p_t(\theta)\hat{A}_t$	no	-	✓
3 $< 1 - \epsilon$	+	$p_t(\theta)\hat{A}_t$	no	+	✓
4 $< 1 - \epsilon$	-	$(1 - \epsilon)\hat{A}_t$	yes	-	0
5 $> 1 + \epsilon$	+	$(1 + \epsilon)\hat{A}_t$	yes	+	0
6 $> 1 + \epsilon$	-	$p_t(\theta)\hat{A}_t$	no	-	✓

表 1: Clipped surrogate objective 的分段情况。

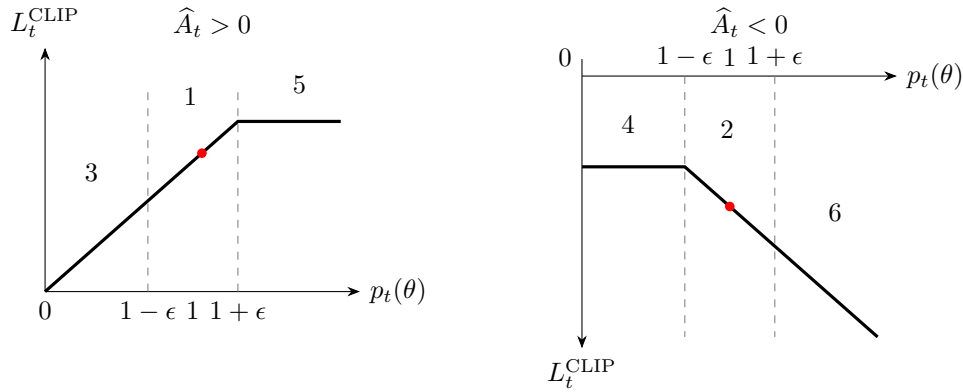


图 1: 按原课件重绘的分段曲线。数字对应表中的六种情况；曲线为示意图。

课件将保留 policy 更新梯度的情况概括为：

- $p_t(\theta)$ 落在邻域 $[1 - \epsilon, 1 + \epsilon]$ 内。
- $p_t(\theta)$ 未落在邻域内，但 advantage 引导 $p_t(\theta)$ 更靠近邻域。

原课件图表署名: Daniel Bick, daniel.bick@live.de, University of Groningen, Faculty of Science and Engineering; *Towards Delivering a Coherent Self-Contained Explanation of Proximal Policy Optimization*。

第五部分 PPO 实现: stable-baselines3

原课件中的源码入口

仓库: <https://github.com/DLR-RM/stable-baselines3>。下列路径均相对于该仓库根目录:

- `stable_baselines3/ppo/`
- `stable_baselines3/common/buffers.py`
- `stable_baselines3/common/evaluation.py`
- `stable_baselines3/common/on_policy_algorithm.py`
- `stable_baselines3/common/policies.py`

25 完整 objective: value loss 与 entropy

对应原课件第 4-8 页。

在前文的 Clipped Surrogate Objective 上加入 value loss 与 entropy term, 得到课件展示的最大化目标:

$$L^{\text{CLIP+VF+S}}(\theta) = \mathbb{E}_t \left[L_t^{\text{CLIP}}(\theta) - c_1 L_t^{\text{VF}}(\theta) + c_2 S[\pi_\theta](s_t) \right]. \quad (168)$$

这里 L_t^{CLIP} 是 policy 的 surrogate objective, L_t^{VF} 是 squared error, $S[\pi_\theta](s_t)$ 是 entropy term。前者提高 policy 的目标值, value loss 使 value network 的估计逼近 target, entropy reward 则用于鼓励 exploration。

25.1 Value target 从哪里来

$$L_t^{\text{VF}}(\theta) = \left(V_{\text{pred}}(s_t) - V_t^{\text{target}} \right)^2, \quad V_t^{\text{target}} = \hat{A}_t + V_{\text{old}}(s_t). \quad (169)$$

- $V_{\text{pred}}(s_t)$ 是当前 value network 的输出。
- V_t^{target} 是由已采集的数据计算出的 return target。
- `advantages` 由 GAE 得到, `values` 是采样时 old value network 的输出。
- 训练时让当前 value network 的输出逼近固定的 target: `returns = advantages + values`。

25.2 连续 action space 中的 exploration

课件以一维 Gaussian distribution 说明标量连续 action 的采样: distribution 由 mean 和 standard deviation 参数化, mean 来自 policy network。课件先讨论将 standard deviation 作为固定 hyperparameter 的情形: 这样若要改变 exploration, 似乎只能手动调整 standard deviation。随后引出可训练的 standard deviation, 在训练中用 stochastic gradient descent 调整。

```
if isinstance(self.action_space, spaces.Box):  
    self.log_std = nn.Parameter(th.zeros(action_dim))
```

这里代码保存的是 `log_std`。原课件引用的相关标题是 *The exploration-exploitation dilemma for adaptive agents*, 作者为 Lilia Rejeb、Zahia Guessoum 和 Rym M'Hallah。

25.3 离散 action space 中的 entropy reward

PPO 依据 action distribution 进行采样。原课件通过两种 distribution 对比 exploration: 若大部分概率集中在一个 action 上, uncertainty 较低; 若可用 actions 的概率更接近均匀分布, uncertainty 更高。

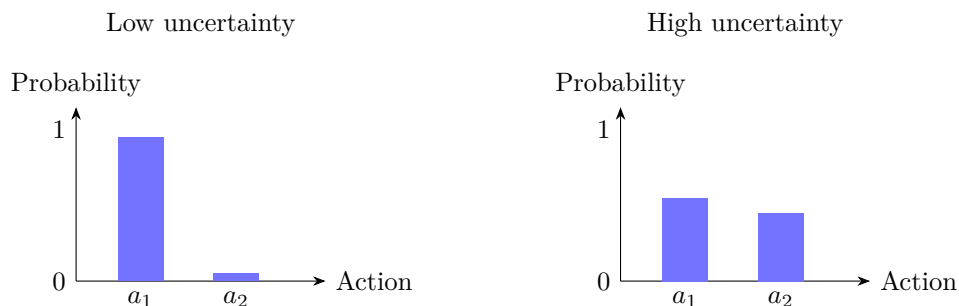


图 2: action distribution 示意图, 柱高仅用于表达概率集中程度。

- 对有限的可用 action 集合分配相同概率, 是课件描述的最大 entropy 情形。
- Entropy reward 鼓励 policy 避免过早把概率集中到少数 actions 上, 从而在遇到的 states 中探索更多不同 actions。
- 课件指出, SB3 的 PPO 在离散与连续 action space 中都包含 entropy term。

26 训练前的 rollout collection

对应原课件第 9–10 页。

1. 用当前 policy 与环境交互, 可以同时多个环境中并行采样。
2. 将每个 timestep 的 observations、actions、rewards、values 和 log probabilities 等数据保存到 RolloutBuffer。
3. 收集完成后计算 GAE 和 returns。
4. 使用这批数据进行多轮 policy 更新, 随后丢弃旧 rollout, 开始新一轮采样。

采样时的“当前 policy”, 在随后的训练阶段被称为 old policy。课件所说的“数据单次使用”, 是指属于同一轮 rollout/update 周期, 并不排除在该周期内进行多个 epochs。

27 RolloutBuffer 与 ReplayBuffer

对应原课件第 10–13 页。

比较项	RolloutBuffer	ReplayBuffer
课件中的用途	PPO 的当轮 rollout	DQN 的 Experience Replay
数据使用	同一轮采样后做多个 epochs，之后丢弃	保留旧数据，训练时随机抽取 batch
写满后的行为	标记 <code>full=True</code> ，不循环覆盖	指针归零，循环覆盖旧数据
取样前提	<code>get()</code> 要求 buffer 已满	有数据时，即使未满也可 <code>sample</code>
取样形式	打乱后分 batch 遍历	随机返回抽中的单个 batch

表 2: 原课件中的两类 buffer 对比。

27.1 固定容量与写入指针

RolloutBuffer 的时间维容量是 `buffer_size = n_steps`。并行环境数由独立的 `n_envs` 维度表示。课件展示的 RolloutBuffer 写入尾部逻辑为：

```
def add(self, obs, action, reward, episode_start, value, log_prob):
    ...
    self.pos += 1
    if self.pos == self.buffer_size:
        self.full = True
```

ReplayBuffer 写满后将指针重新置为零：

```
self.pos += 1
if self.pos == self.buffer_size:
    self.full = True
    self.pos = 0
```

RolloutBuffer 取样前要求填满：

```
def get(self, batch_size=None):
    assert self.full, ""
    ...
```

27.2 Reset 与 ReplayBuffer 的随机取样

RolloutBuffer 重置时，将各数组重新初始化，并重置 generator 状态；其父类 BaseBuffer 的 reset 负责重置写入指针和 full 标记。

RolloutBuffer 的 reset 节选：

```
def reset(self) -> None:
    self.observations = np.zeros(...)
    self.actions = np.zeros(...)
    ...
    self.generator_ready = False
    super().reset()
```

BaseBuffer 的 reset：

```
def reset(self) -> None:
    self.pos = 0
    self.full = False
```

课件展示的 ReplayBuffer 取样逻辑：

```
def sample(self, batch_size: int, env=None):
    upper_bound = self.buffer_size if self.full else self.pos
    batch_inds = np.random.randint(0, upper_bound, size=batch_size)
    return self._get_samples(batch_inds)
```

未满时只从已写入的范围随机抽取；满时可从整个有效容量抽取。这与 RolloutBuffer 的 get() 先要求 full=True 的行为不同。

27.3 Trajectory 与 buffer 边界

原课件将 RolloutBuffer 描述为保存 trajectory 数据。结合后面的 GAE 代码，应区分固定长度的 rollout 与一个完整 episode：buffer 内可能包含多条 episodes 的片段，最后一个 episode 也可能尚未结束。因此，需要通过 episode boundary mask 和最后状态的 value estimate 处理边界。

27.4 RolloutBuffer 中存放的字段

对应原课件第 14 页。

```
def reset(self) -> None:
    self.observations = np.zeros(
        (self.buffer_size, self.n_envs, *self.obs_shape),
        dtype=self.observation_space.dtype,
    )
    self.actions = np.zeros(
        (self.buffer_size, self.n_envs, self.action_dim),
        dtype=self.action_space.dtype,
    )
    self.rewards = np.zeros(
        (self.buffer_size, self.n_envs), dtype=np.float32
    )
    self.returns = np.zeros(
        (self.buffer_size, self.n_envs), dtype=np.float32
    )
    self.episode_starts = np.zeros(
        (self.buffer_size, self.n_envs), dtype=np.float32
    )
    self.values = np.zeros(
        (self.buffer_size, self.n_envs), dtype=np.float32
    )
    self.log_probs = np.zeros(
        (self.buffer_size, self.n_envs), dtype=np.float32
    )
    self.advantages = np.zeros(
        (self.buffer_size, self.n_envs), dtype=np.float32
    )
    self.generator_ready = False
    super().reset()
```

- `observations` 与 `actions` 除时间、环境两个维度外，还保留 `observation shape` 与 `action dimension`。
- `rewards`、`values`、`log_probs` 和 `episode_starts` 保存采集时的数据。
- `advantages` 和 `returns` 在采样后计算，而不是直接从环境获得。
- 对每个 `timestep`、每个环境，均有 `returns = advantages + values`。

28 Generalized Advantage Estimation

对应原课件第 15–18 页。

课件沿时间维反向计算 GAE。记 m_t 为代码中的 `next_non_terminal`，则该代码对应的递推为

$$\delta_t = r_t + \gamma m_t V_{\text{old}}(s_{t+1}) - V_{\text{old}}(s_t), \quad (170)$$

$$\hat{A}_t = \delta_t + \gamma \lambda m_t \hat{A}_{t+1}, \quad (171)$$

$$\hat{R}_t = \hat{A}_t + V_{\text{old}}(s_t). \quad (172)$$

本节的 r_t 表示 reward，不是前文带有参数 θ 的 probability ratio $r_t(\theta)$ 。以下代码保留课件的 GAE 实现逻辑，将原截图注释移入正文：

```
def compute_returns_and_advantage(
    self, last_values: th.Tensor, dones: np.ndarray
) -> None:
    last_values = last_values.clone().cpu().numpy().flatten()
    last_gae_lam = 0
    for step in reversed(range(self.buffer_size)):
        if step == self.buffer_size - 1:
            next_non_terminal = 1.0 - dones.astype(np.float32)
            next_values = last_values
        else:
            next_non_terminal = 1.0 - self.episode_starts[step + 1]
            next_values = self.values[step + 1]
        delta = (
            self.rewards[step]
            + self.gamma * next_values * next_non_terminal
            - self.values[step]
        )
        last_gae_lam = (
            delta
            + self.gamma * self.gae_lambda
            * next_non_terminal * last_gae_lam
        )
        self.advantages[step] = last_gae_lam
    self.returns = self.advantages + self.values
```

`last_values` 提供 buffer 后一个状态的 value estimate，用于 bootstrapping。反向递推开始时，`last_gae_lam` 设为零；每一步先计算 TD residual `delta`，再把后续 advantage 按 $\gamma\lambda$ 折扣累加。

28.1 最后一步使用 `dones`，内部使用 `episode_starts`

在反向计算的起始步，即 `step == self.buffer_size - 1` 时，代码读取传入的 `dones`：

```
next_non_terminal = 1.0 - dones.astype(np.float32)
next_values = last_values
```

按该代码的 `mask` 逻辑，如果 `episode` 继续，则使用 $r_t + \gamma V(s_{t+1}) - V(s_t)$ ；如果 `mask` 为零，则只留下 $r_t - V(s_t)$ 。这里“`dones` 只作用于最后一步”是指本函数传入的末步标记，并不是说 `buffer` 内不存在其他 `episode` 终点。

对 `buffer` 内其他 `timesteps`，边界由下一个位置的 `episode_starts` 给出：

```
next_non_terminal = 1.0 - self.episode_starts[step + 1]
next_values = self.values[step + 1]
```

- `episode_starts[step + 1] == 1` 表示下一条数据属于新 `episode`。
- 此时 `next_non_terminal == 0`，阻止下一条 `episode` 的 `value` 和 `advantage` 混入当前 `episode` 的递推。
- 原课件称其为阻断“反向传播”；这里具体指 GAE 的反向时间递推，不是 `neural network` 的 `autograd backward`。
- 同一 `episode` 内的连续 `transition` 不需截断递推；固定长度 `buffer` 的尾部则使用 `last_values` 进行 `bootstrapping`。

28.2 两个环境、三个 `timesteps` 的示例

对应原课件第 19 页；`buffer_size=3`，`n_envs=2`。

```
dones = np.array([
    [0., 0.],
    [1., 0.],
    [0., 0.],
], dtype=np.float32)

episode_starts = np.array([
    [1., 1.],
    [0., 0.],
    [1., 0.],
], dtype=np.float32)
```

在该示例的内部相邻 `timesteps`，`episode_starts[t+1] == dones[t]`。初始行表示起始状态；最后一行 `dones` 对应的下一行起始标记在当前 `buffer` 之外。课件建议在 `stable_baselines3/common/on_policy_algorithm.py` 中查看这些字段是如何产生的。

28.3 仅保存 dones 的 GAE 写法

对应原课件第 20 页。

如果每一步的 done 标记都被保存，并与 transitions 正确对齐，就可以直接使用 dones 表示相同的 episode 边界。原课件引用 labml 的实现作为例子。下面按截图整理其计算主体，把截图中的循环变量统一写为 step，省略类型标注与初始化上下文：

```
def __call__(self, done, rewards, values):
    advantages = np.zeros(
        (self.n_workers, self.worker_steps), dtype=np.float32
    )
    last_advantage = 0
    last_value = values[:, -1]
    for step in reversed(range(self.worker_steps)):
        mask = 1.0 - done[:, step]
        last_value = last_value * mask
        last_advantage = last_advantage * mask
        delta = (
            rewards[:, step]
            + self.gamma * last_value
            - values[:, step]
        )
        last_advantage = (
            delta + self.gamma * self.lambda_ * last_advantage
        )
        advantages[:, step] = last_advantage
        last_value = values[:, step]
    return advantages
```

这里的数组采用 [worker, step] 的索引顺序，与前面的 [step, env] 不同；values 还包含末尾用于 bootstrapping 的 value。两种写法的共同点是使用 mask 阻止 advantage 累加跨越 episode 边界。

原课件引用：[labmlai / annotated_deep_learning_paper_implementations](#)，文件 [labml_nn/rl/ppo/gae.py](#)。原课件还展示了“GAE 的原理、代码实现：Generalized Advantage Estimation”的讲解截图。

29 PPO 的训练流程与 schedules

对应原课件第 21–24 页。

课件展示的 `train()` 开始部分如下：

```
def train(self) -> None:
    self.policy.set_training_mode(True)
    self._update_learning_rate(self.policy.optimizer)
    clip_range = self.clip_range(self._current_progress_remaining)
    if self.clip_range_vf is not None:
        clip_range_vf = self.clip_range_vf(
            self._current_progress_remaining
        )
    ...
```

29.1 Learning rate 与 progress remaining

```
self._current_progress_remaining = (
    1.0 - float(num_timesteps) / float(total_timesteps)
)
```

Learning rate 可以与 `progress_remaining` 挂钩，例如线性衰减，也可以保持常数，或采用其他 schedule。

29.2 clip_range 与 clip_range_vf

- `clip_range` 是 policy ratio 的 clipping 参数，可以是训练进度的函数。原课件展示的默认值为常数 0.2，也可以设置为衰减形式。
- `clip_range_vf` 限制 value function 的变化量，防止 value estimate 的单次改动过大。
- 原课件把 value clipping 标注为 “specific to the OpenAI implementation”，并指出其展示的默认设置不启用该功能；启用后效果不一定更好。

课件用于说明 clipping 范围的简化片段是：

```
ratio = new_policy_prob / old_policy_prob
ratio = th.clamp(ratio, 1 - clip_range, 1 + clip_range)
```

这只是单独展示 `clamp` 操作；实际 clipped surrogate loss 仍同时保留未裁剪与已裁剪的两个分支，再取最小值。

29.3 Epochs 与 minibatch shuffle

```
for epoch in range(self.n_epochs):
    approx_kl_divs = []
    for rollout_data in self.rollout_buffer.get(self.batch_size):
        ...
```

原课件展示的 `n_epochs` 默认值为 10。每个 epoch 从当前 rollout 中取 minibatches。课件展示的 `get()` 核心逻辑为随机排列索引，再分块返回：

```
def get(self, batch_size):
    indices = np.random.permutation(self.buffer_size * self.n_envs)
    start_idx = 0
    while start_idx < self.buffer_size * self.n_envs:
        yield self._get_samples(
            indices[start_idx : start_idx + batch_size]
        )
        start_idx += batch_size
```

此片段仅整理索引循环，按课件意图补全 `start_idx` 的初始化、循环边界与递增；原函数的其他准备步骤未在此重复。

- 采集和计算 GAE 时，数据应保留时间顺序，才能按 trajectory 递推 advantage。
- 训练时，每个样本的 advantage 与 return 已经算好，因此可以打乱样本次序。
- 多个 epochs 复用的是当前这批 rollout，而不是不断累积历史 replay 数据。

30 用新 policy 重新评估旧样本

对应原课件第 25 页。

```
values, log_prob, entropy = self.policy.evaluate_actions(
    rollout_data.observations, actions
)
```

当前 policy 对已经采集的 observations 和 actions 重新计算 values、log probabilities 与 entropy。这里不是重新采样一个 action，而是评估旧 action 在当前 distribution 下的 likelihood：

```
log_prob = distribution.log_prob(actions)
```

新的 `log_prob` 随后与采样时保存的 `old_log_prob` 一起用于计算 importance sampling ratio。

31 Advantage normalization 与 probability ratio

对应原课件第 26-27 页。

31.1 在每个 minibatch 内归一化 advantages

```
advantages = rollout_data.advantages
if self.normalize_advantage and len(advantages) > 1:
    advantages = (
        (advantages - advantages.mean())
        / (advantages.std() + 1e-8)
    )
```

课件指出, 某些 advantage 的数值可能过大或过小, 因此在每个 minibatch 内部进行 normalization。对应的形式为

$$\tilde{A}_t = \frac{\hat{A}_t - \text{mean}_{\mathcal{B}}(\hat{A})}{\text{std}_{\mathcal{B}}(\hat{A}) + 10^{-8}}, \quad (173)$$

其中 $\mathcal{B}$ 表示当前 minibatch。当 minibatch 只有一个样本时, 课件代码直接跳过 normalization, 不依赖单样本 standard deviation 的数值约定。

31.2 由 log probabilities 得到 ratio

```
ratio = th.exp(log_prob - rollout_data.old_log_prob)
```

$$\begin{aligned} p_t(\theta) &= \exp(\log \pi_{\theta}(a_t | s_t) - \log \pi_{\theta_{\text{old}}}(a_t | s_t)) \\ &= \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}. \end{aligned} \quad (174)$$

- Ratio 是 importance weight, 比较 new policy 与 old policy 对同一个 action 的 likelihood。
- 直观地说, 它描述新 policy 认为这个 action 比原来更可能了多少。
- 对连续 action space, 这里的 likelihood 使用 probability density; 对离散 action space, 使用 action probability。
- PPO-clip 在 surrogate objective 中对该 ratio 做 clipping, 而不是对 policy 参数直接施加 hard constraint。

32 Clipped surrogate loss 与 value clipping

对应原课件第 28-29 页。

32.1 Policy loss 的两个分支

```
policy_loss_1 = advantages * ratio
policy_loss_2 = advantages * th.clamp(
    ratio, 1 - clip_range, 1 + clip_range
)
policy_loss = -th.min(policy_loss_1, policy_loss_2).mean()

pg_losses.append(policy_loss.item())
clip_fraction = th.mean(
    (th.abs(ratio - 1) > clip_range).float()
).item()
clip_fractions.append(clip_fraction)
```

负号把要最大化的 clipped surrogate objective 转为要最小化的 loss。clip_fraction 记录 ratio 落在 clipping 区间之外的样本比例：

$$\text{clip_fraction} = \frac{1}{|\mathcal{B}|} \sum_{t \in \mathcal{B}} \mathbf{1}\{|p_t(\theta) - 1| > \epsilon\}. \quad (175)$$

它只按 ratio 判断是否越界，不等同于前文六种情况中“最终取到平坦分支、梯度为零”的样本比例。

32.2 限制 value function 的变化量

```
if self.clip_range_vf is None:
    values_pred = values
else:
    values_pred = rollout_data.old_values + th.clamp(
        values - rollout_data.old_values,
        -clip_range_vf,
        clip_range_vf,
    )
```

设 ϵ_V 表示 clip_range_vf，则启用该功能时，

$$V_{\text{pred}}(s_t) = V_{\text{old}}(s_t) + \text{clip}(V_{\theta}(s_t) - V_{\text{old}}(s_t), -\epsilon_V, \epsilon_V). \quad (176)$$

裁剪的是相对于 old value 的变化量，不是 value 本身的绝对范围。未启用时直接使用当前 values；之后的 squared error target 仍是式 (169) 中由 rollout 计算出的 return。

33 KL early stopping

对应原课件第 30 页。

课件通过近似 KL divergence 衡量 new policy 与 old policy 的差异。当配置了 `target_kl`，且估计值超过其 1.5 倍时，设置 `continue_training=False` 并退出当前 minibatch 循环：

```
with th.no_grad():
    log_ratio = log_prob - rollout_data.old_log_prob
    approx_kl_div = th.mean(
        (th.exp(log_ratio) - 1) - log_ratio
    ).cpu().numpy()
    approx_kl_divs.append(approx_kl_div)

if (
    self.target_kl is not None
    and approx_kl_div > 1.5 * self.target_kl
):
    continue_training = False
    if self.verbose >= 1:
        print(
            f"Early stopping at step {epoch} due to reaching "
            f"max kl: {approx_kl_div:.2f}"
        )
    break
```

该片段位于训练循环内部。课件将其用途概括为：当 policy 差异过大时停止继续更新，防止更新幅度过大。`break` 本身只退出当前所在循环；`continue_training` 是供外层训练流程使用的停止标志。

33.1 与 PPO-clip 的关系

前文的 clipping 作用于 objective 中的 ratio 分支，而这里的 early stopping 使用整批样本的近似 KL 作为额外判断。因此，`clip_range` 与 `target_kl` 在代码中承担不同作用：一个构造 loss，一个决定是否继续本轮训练。

34 KL divergence 的 Monte Carlo estimation

对应原课件第 31 页。

课件最后给出使用 $r - 1 - \log r$ 的 KL estimator。沿用本文的 probability ratio 记号 $p_t(\theta)$ ，代码中的 batch 估计为

$$\hat{D}_{\text{KL}} = \frac{1}{|\mathcal{B}|} \sum_{t \in \mathcal{B}} [p_t(\theta) - 1 - \log p_t(\theta)]. \quad (177)$$

它与 `log_ratio` 的实现逐项对应：

$$\begin{aligned} \text{log_ratio} &= \log p_t(\theta), \\ \text{th.exp(log_ratio)} &= p_t(\theta), \\ \text{approx_kl_div} &= \text{mean}_{\mathcal{B}} [p_t(\theta) - 1 - \log p_t(\theta)]. \end{aligned}$$

原课件将这种 estimator 的特点概括为“无偏、方差小”，并指出 DeepSeek GRPO 也使用了类似的估计形式。这里保留课件的说明，不把“方差小”改写为对任意 distribution 都成立的比较结论，也不将单个 minibatch 的估计值视为精确 KL。

原课件引用：John Schulman, *Approximating KL Divergence*。

<http://joschu.net/blog/kl-approx.html>

本次转写保留该来源地址，没有复制网页正文或个人主页介绍。

附录：来源、索引与校订

A 来源与整理说明

A.1 手写笔记

来源与范围：手写笔记部分依据项目中的 `prism-uploads/强化学习_202609161755_08767.pdf` 逐页整理。PDF 共 27 页，第 1-26 页有内容，第 27 页为空白。手写正文、公式推导、彩色批注和嵌入的算法截图均转为可编辑内容；横线背景、荧光标记和重复书写不再逐笔复刻。手写笔记来自用户提供的笔记 PDF，与 PPO 课件的作者署名分开标注。

整理约定：按基础知识、Policy Gradient、TRPO、PPO 和实现的学习顺序编排，以中文连接和润色说明，英文专业术语仍予保留。各节标注原 PDF 页码，便于对照。能由上下文确定的符号笔误已修正；涉及结论成立条件的地方用 **校订说明** 区分，不把补充条件冒充原稿内容。本附录列出主要校订和页码对应关系。

A.2 PPO 课件与实现

PPO 理论部分整理自 `PP0.pdf`；PPO 实现部分整理自 `PP0-SB3.pdf`。原课件作者：东川路第一可爱猫猫虫。

PPO 实现部分根据 `PP0-SB3.pdf` 的全部 31 页整理。重复出现的代码片段按主题合并，截图中的代码、公式和数据示例改为可编辑文本，entropy 示意图用 TikZ 重绘。装饰图片、网页界面与个人主页截图不再嵌入。下文的默认参数和实现细节均指课件展示的版本；课件没有提供确定的 release 或 commit，因此这里不将其表述为最新 SB3 实现。

代码是按原课件整理的节选，而不是一个独立可运行的完整训练脚本。其中 `th`、`np`、`nn` 分别沿用课件中的 PyTorch、NumPy 和 neural network 模块别名；省略号表示未展示的上下文。

B 原笔记页码索引与校订记录

B.1 完整页码对应关系

原 PDF 页码	已转写内容
1	RL 目标、统计估计、MDP 与一步交互
2–3	Return、Bellman expectation、矩阵求解、action value、contraction
4	最优 value/policy 的证明、reward affine transformation
5–6	Value iteration、policy iteration、truncated policy iteration 与算法截图
7–8	MC basic、Exploring Starts、first/every visit、epsilon-greedy、均值递推
9–10	Robbins–Monro、SGD、相对 gradient noise、batch/minibatch、TD 引入
11–12	TD error、bootstrapping、step size、SARSA 更新与算法
13–14	n-step SARSA、Q-learning、on/off-policy、行为与目标 policy
15–16	Function approximation、stationary weighting、SARSA 近似、DQN 与 replay
17–18	Average value/reward、长期平均证明、policy gradient 与 visitation weighting
19	Gradient update 直觉、REINFORCE、Q Actor–Critic
20–21	Baseline 零期望、covariance trace、最优 baseline、advantage、A2C、逆变换采样
22	Importance sampling、off-policy actor、DPG、DDPG 阅读线索
23	TRPO、Performance Difference Lemma 与 discounted visitation
24	Surrogate 一阶一致性、performance bound、TV、log-sum、Pinsker
25	Pinsker 结尾、KL penalty/constraint、importance ratio、Taylor approximation
26	Gradient 代码、Lagrangian、局部最优更新、CG、Hessian–vector product
27	空白横线页，无需转写正文

B.2 主要校订及原稿边界

1. 补齐 Bellman 公式中漏写的 γ ，统一即时 reward、return 和时间下标。
2. Reward 缩放保持最优 policy 的条件由非零改为正数，并说明加常数的时域条件。
3. 区分条件期望 Bellman residual 为零与单样本 TD error 为零；DQN 的随机平方 residual 也不必为零。
4. 修正 value approximation 的 SGD 符号，以及 epsilon-greedy 非 greedy 分支的概率。
5. 补充 stochastic approximation 的 noise 控制条件；不把常数 step size 与递减 step size 的收敛结论混为一谈。
6. 区分固定初始分布、stationary distribution 和 discounted visitation frequency；显式说明归一化系数、average reward 与 discounted action value 的区别。

7. REINFORCE 的整理算法明确固定采样参数与外层时间折扣，不沿用原截图中混杂的 episode/time-step 参数下标。
8. 修正 importance sampling 的易采样分布标记；action ratio 不自动修正 state distribution, off-policy 局部方向不等于任意 objective 的精确 gradient。
9. Pinsker 证明末行改为 $2D_{TV}^2$ ，区分 TRPO 最大 KL 理论保证与平均 KL、采样和局部近似的实际算法。
10. 原稿只列出 DDPG 关键词，并未展开其算法；TRPO 末尾也未展开 line search。这些内容不按“已经识别出的手写正文”追加。

B.3 用于核对关键条件的参考资料

以下资料仅用于校订 policy-gradient 的目标与分布约定、DPG 的 off-policy 区分，以及 TRPO 理论界与实际近似的关系；手写笔记正文仍来自所提供的 PDF。

1. Sutton 等, *Policy Gradient Methods for Reinforcement Learning with Function Approximation*, 1999。 [论文原文](#)。
2. Silver 等, *Deterministic Policy Gradient Algorithms*, 2014。 [论文原文](#)。
3. Schulman 等, *Trust Region Policy Optimization*, 2015。 [论文原文](#)。

C PPO 实现与原课件的页码对应

原课件页码	本文内容
1-3	标题、内容提要与源码入口
4-8	完整 objective、value target 与 entropy exploration
9-14	Rollout collection、buffer 对比、reset 与数据字段
15-20	GAE 递推、episode mask、双环境示例与 labml 写法
21-24	训练入口、schedules、epochs 与 minibatch shuffle
25-27	Action evaluation、advantage normalization 与 ratio
28-29	Clipped surrogate loss 与 value clipping
30-31	KL early stopping 与 Monte Carlo estimator

[返回完整目录](#)